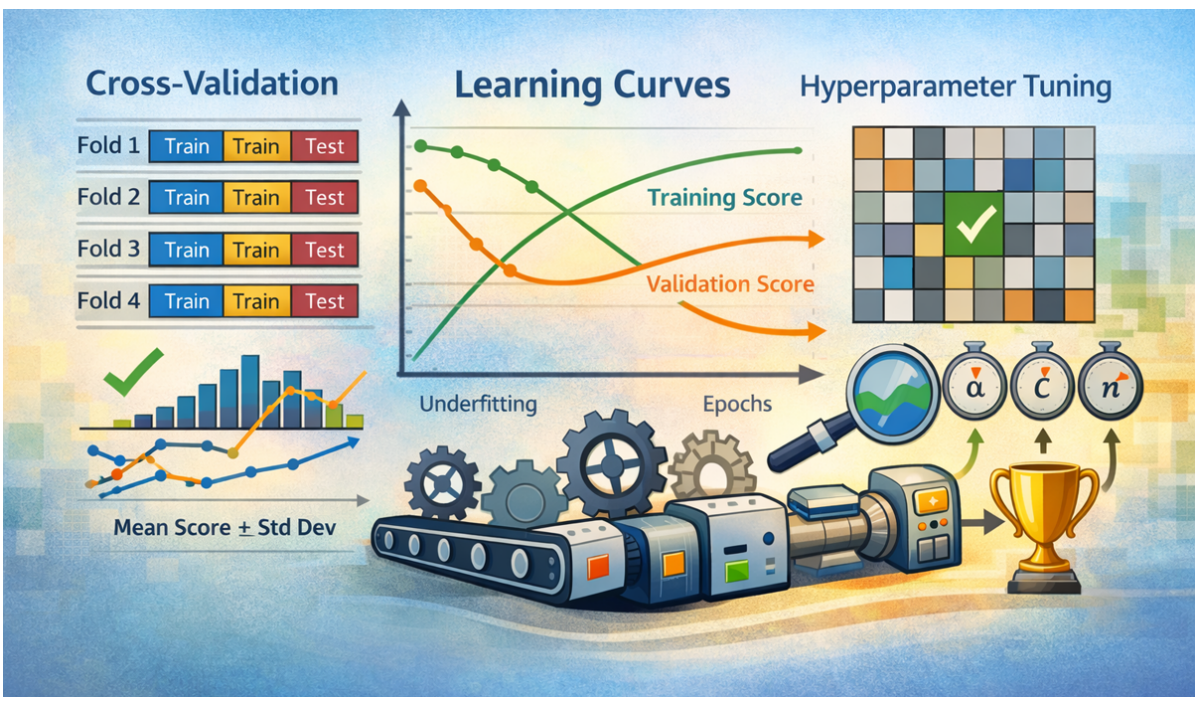
Chapter 15: Model Evaluation, Selection & Tuning

<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to implement multiple cross-validation strategies (K-Fold, Stratified K-Fold, GroupKFold, TimeSeriesSplit) appropriate for different data structures <{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to use learning curves and validation curves to diagnose underfitting, overfitting, and hyperparameter sensitivity <{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to perform systematic hyperparameter tuning using GridSearchCV and RandomizedSearchCV within computational budgets <{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to set up nested cross-validation to obtain unbiased estimates of final model performance <{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to compare multiple algorithms fairly using consistent evaluation methodology and justify final model selection decisions

15.1Introduction



Chapter philosophy

Up to this point, you have learned how to build models, engineer predictors, and evaluate results using an 80/20 train/test split and common performance metrics. The next step is learning how to evaluate and compare models the way professionals do it: using cross-validation to get reliable estimates, using diagnostic curves to understand *why* performance changes, and using systematic tuning methods to choose settings without accidentally overfitting to a single split.

The key idea is that model evaluation is not only about getting a good number once. Professional model development emphasizes **reliable estimates** of performance, **fair comparisons** across competing algorithms, and **repeatable selection rules** that avoid accidental overfitting to a particular train/test split.

Learning objectives

- Explain why a single train/test split can be misleading for model evaluation.
- Implement cross-validation strategies appropriate for regression and classification problems.
- Use learning curves and validation curves to diagnose underfitting, overfitting, and hyperparameter sensitivity.
- Apply hyperparameter tuning using *GridSearchCV* and *RandomizedSearchCV* with a clear compute budget.
- Compare multiple algorithms fairly and justify a final model choice using performance metrics, variance, and practical constraints.
- Balance performance, interpretability, and probability quality when selecting models for deployment.

Prerequisites recap

This chapter assumes you already know how to build models and interpret basic evaluation results from earlier chapters. The main shift in this chapter is learning how to make evaluation and selection *reliable* and *repeatable*.

- Regression evaluation metrics (MAE, RMSE, R^2).
- Train/test splits and basic overfitting detection.
- Decision tree evaluation and the role of model complexity.
- Classification evaluation metrics (accuracy, precision, recall, F1, ROC AUC, log loss, confusion matrices).
- Ensembles and feature importance at a conceptual level.

Three-pass learning approach

This chapter is organized using a three-pass approach. Each pass builds on the previous one, and each pass is useful on its own. After Pass 1, you can evaluate models reliably. After Pass 2, you can diagnose why a model is performing the way it is. After Pass 3, you can tune and select models systematically.

1. **Pass 1: Reliable evaluation.** Learn cross-validation, scoring, and pipeline-based evaluation so performance estimates are stable and comparisons are fair.
2. **Pass 2: Diagnosis tools.** Use learning curves and validation curves to identify underfitting, overfitting, and hyperparameter sensitivity.
3. **Pass 3: Selection and tuning.** Use grid search and randomized search to tune hyperparameters within a compute budget, then compare models using consistent evaluation rules. Nested cross-validation is included as an optional advanced method for rigorous reporting.

A comprehensive set of links to the scikit-learn documentation for supervised learning algorithms and related topics is included at the end of this chapter as a reference appendix.

15.2Cross-Validation

Why train/test isn't enough

A single train/test split is useful for learning and quick iteration, but it produces a single estimate of performance that can change noticeably depending on which samples land in the test set. This problem is especially noticeable when datasets are small or when the positive class is rare. Cross-validation is a resampling-based evaluation technique that repeatedly splits the training data into different train/validation folds to produce multiple performance estimates, allowing you to measure both typical model performance and its variability. By producing a distribution of scores across multiple folds, cross-validation lets you report both the typical performance (mean) and the variability (standard deviation).

In this chapter, the goal is not to memorize every evaluation function in scikit-learn. Instead, the goal is to build a repeatable professional workflow: freeze an untouched test set, evaluate models reliably with cross-validation on the training data, diagnose bias and variance, and only then tune hyperparameters and compare algorithms fairly.

Three-way split vs cross-validation

A common question is whether to use a three-way split (train/validation/test) or cross-validation. A three-way split is simple and can work well for very large datasets, but cross-validation is usually more reliable for small and

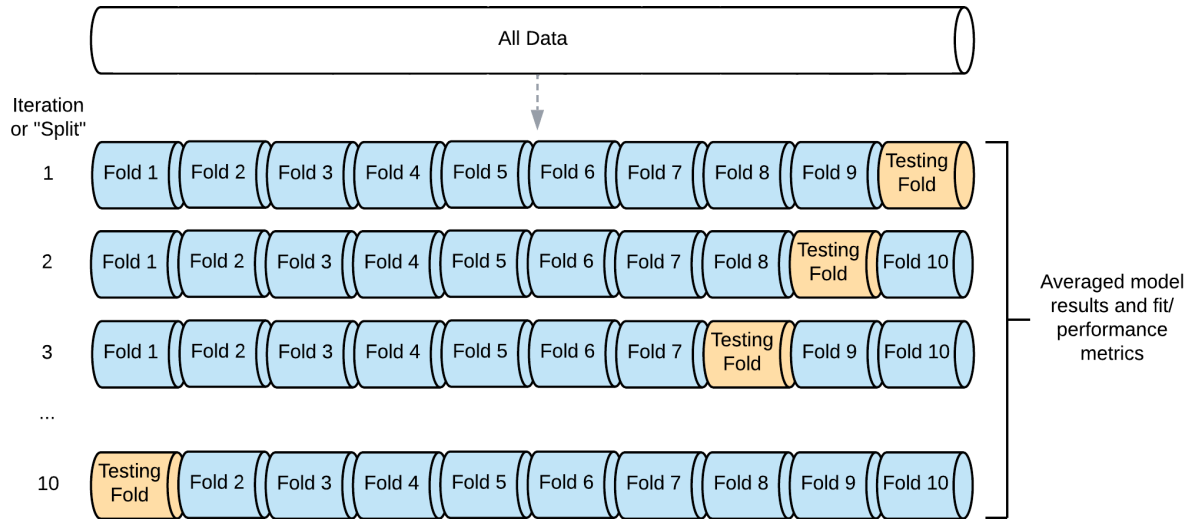
medium datasets because it reduces the chance that an “unlucky” split produces misleading results.

A practical decision rule is:

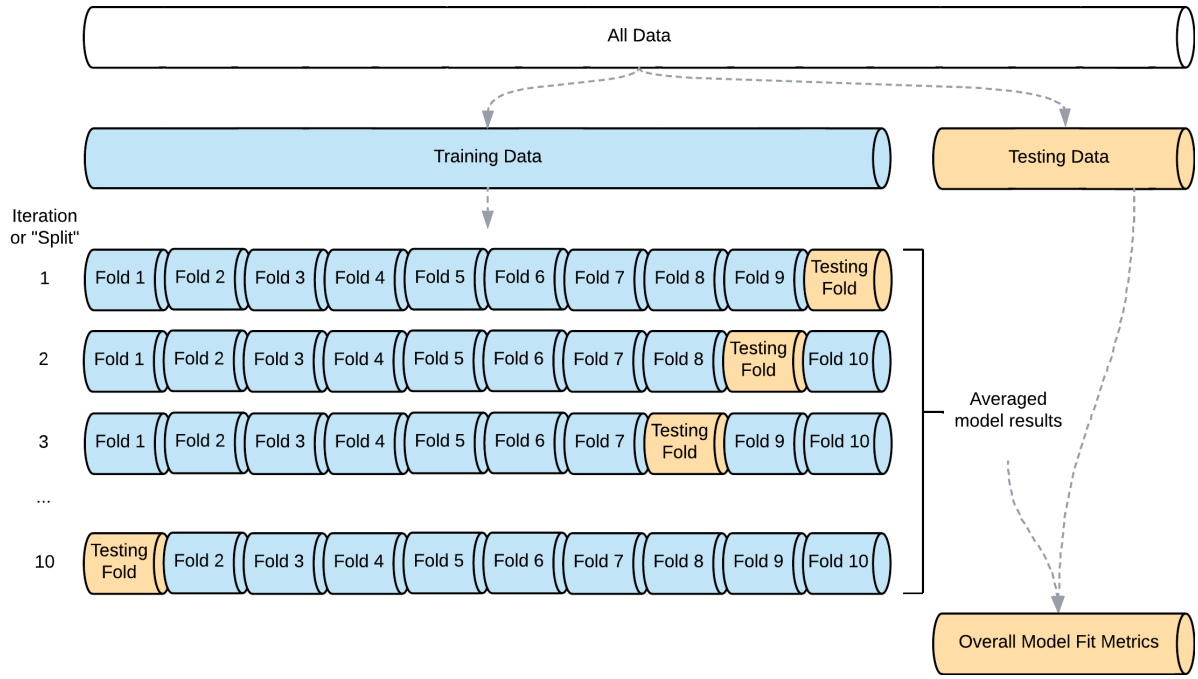
- **Small/medium datasets:** freeze a final test set once, then use cross-validation on the training data for model selection and tuning.
- **Very large datasets:** train/validation/test splits can be sufficient; cross-validation may be computationally unnecessary.
- **Key principle:** never use the test set for model selection or hyperparameter tuning. The test set is for final evaluation only.

Cross-validation foundations

Cross-validation divides the available training data into multiple “folds.” Each fold takes a turn being the validation (testing) fold while the remaining folds are used to train the model. The model is fit multiple times and evaluated multiple times, producing a set of scores that can be summarized with an average and standard deviation.



In professional workflows, cross-validation is typically used on the training set only. The test set remains untouched until the end, when you evaluate the final selected model one time. This reduces “evaluation leakage” and makes your final report more trustworthy.



The remainder of this section uses the same Lending Club dataset used earlier (lc_small.csv) so students can focus on evaluation mechanics rather than constantly switching datasets. The code is organized to create reusable objects (data cleaning, preprocessing, model pipelines, and scoring dictionaries) that persist across later sections of the chapter.

Cross-validation in practice

Start by loading and cleaning the Lending Club dataset using the same cleaning steps from the prior chapter. This produces a modeling-ready DataFrame and keeps the workflow consistent across chapters.

```
import numpy as np
import pandas as pd

SEED = 27
DATA_PATH = &quot;/content/drive/MyDrive/Colab Notebooks/data/lc_small.csv&quot;
df = pd.read_csv(DATA_PATH)

# Drop columns used in prior chapter
drop_cols = [&quot;loan_status_numeric&quot;, &quot;emp_title&quot;,
&quot;title&quot;]
drop_cols = [c for c in drop_cols if c in df.columns]
df = df.drop(columns=drop_cols)

# Convert issue date to &quot;age&quot; feature (days since most recent issue date)
if &quot;issue_d&quot; in df.columns:
  df[&quot;issue_d&quot;] = pd.to_datetime(df[&quot;issue_d&quot;],
errors=&quot;coerce&quot;)
  max_issue_date = df[&quot;issue_d&quot;].max()
  df[&quot;issue_age_days&quot;] = (max_issue_date - df[&quot;issue_d&quot;]).dt.days
  df = df.drop(columns=[&quot;issue_d&quot;])

# Parse term to numeric months
if &quot;term&quot; in df.columns:
  df[&quot;term&quot;] =
df[&quot;term&quot;].astype(str).str.strip().str.extract(r&quot;(\\d+)&quot;).astype(float)

# Parse employment length to numeric years
if &quot;emp_length&quot; in df.columns:
  emp = df[&quot;emp_length&quot;].astype(str).str.strip()
  emp = emp.replace({&quot;nan&quot;: np.nan, &quot;None&quot;: np.nan})
  emp = emp.replace({&quot;10+ years&quot;: &quot;10&quot;, &quot;&lt; 1 year&quot;:
&quot;0&quot;})
  emp = emp.str.extract(r&quot;(\\d+)&quot;)[0]
  df[&quot;emp_length_years&quot;] = pd.to_numeric(emp, errors=&quot;coerce&quot;)
  df = df.drop(columns=[&quot;emp_length&quot;])

# Missingness indicators for selected delinquency/record columns
for col in [&quot;mths_since_last_delinq&quot;, &quot;mths_since_last_record&quot;]:
  if col in df.columns:
    ind_col = col + &quot;_missing&quot;
    df[ind_col] = df[col].isna().astype(int)
    max_val = df[col].max(skipna=True)
    fill_val = (max_val + 1) if pd.notna(max_val) else 0
    df[col] = df[col].fillna(fill_val)

df.shape


# Output: (10476, 34)
```

Next, define the target variable *y*. In most Lending Club teaching datasets, the target is a loan outcome column such as *loan_status*. The code below uses a safe pattern: it looks for a plausible target column, then converts it to a binary label suitable for classification.

```
# Target definition (binary): 1 = good, 0 = bad
bad_statuses = {&quot;Charged Off&quot;, &quot;Default&quot;}
df[&quot;loan_good&quot;] =
(~df[&quot;loan_status&quot;].isin(bad_statuses)).astype(int)
y = df[&quot;loan_good&quot;].copy()
X = df.drop(columns=[&quot;loan_status&quot;, &quot;loan_good&quot;]).copy()
X.shape, y.value_counts()


# Output:
# ((10476, 33),
#  loan_good
#  1    9578
#  0     898
#  Name: count, dtype: int64)
```

Freeze an untouched test set once. All cross-validation and tuning should happen on the training portion only. This mirrors a standard professional workflow.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
  X,
  y,
  test_size=0.20,
  random_state=SEED,
  stratify=y
)

X_train.shape, X_test.shape


# Output: ((8380, 33), (2096, 33))
```

K-Fold cross-validation

K-Fold cross-validation divides the *training data* into k approximately equal-sized folds. The model is trained k separate times. In each iteration, one fold is held out as the validation fold, while the remaining $k - 1$ folds are used for training. This process produces k *independent validation scores*, one from each fold.

Rather than relying on a single train/test split, K-Fold cross-validation estimates model performance by averaging results across all folds. This provides a more reliable estimate of how the model is expected to perform on unseen data and allows you to report both the *mean performance* and its *variability* (standard deviation).

Cross-validation replaces the *validation set*, not the test set. During model development, you should never tune hyperparameters (you'll learn more about this later) or compare models using the test set. Instead, K-Fold cross-validation allows you to reuse the training data efficiently while still obtaining an honest estimate of model performance.

```
from sklearn.model_selection import KFold

# Five-fold cross-validation with shuffling for randomness
kfold = KFold(
  n_splits=5,
  shuffle=True,
  random_state=SEED
)

kfold


# Output: KFold(n_splits=5, random_state=27, shuffle=True)
```

Stratified K-Fold (classification)

Stratified K-Fold cross-validation is a classification-focused variant of K-Fold that explicitly preserves the *class distribution* in every fold. Each fold contains approximately the same proportion of each class as the full training dataset, ensuring that both majority and minority classes are represented during every validation step.

This distinction is critical for imbalanced classification problems. With a standard K-Fold split, it is possible for some folds to contain very few—or even zero—examples of a rare class. When that happens, performance metrics such as recall, precision, ROC AUC, or log loss can become unstable or misleading, because the model is never evaluated on meaningful examples of the minority class.

Stratified K-Fold avoids this problem by enforcing class balance across folds. Each validation fold becomes a small but representative version of the full training set, allowing the model to be evaluated consistently on all classes. As a result, cross-validation scores are more stable, more comparable across folds, and more indicative of how the model will behave in real-world deployment.

The recommended workflow remains unchanged from standard K-Fold: stratified cross-validation is applied to the *training set only*, while the test set is held out and used exactly once for final evaluation. Stratification does not change the mechanics of cross-validation—it improves the quality and reliability of the validation signal for classification tasks.

```
from sklearn.model_selection import StratifiedKFold

# Stratified five-fold cross-validation preserves class balance in each fold
skf = StratifiedKFold(
  n_splits=5,
  shuffle=True,
  random_state=SEED
)

skf


# Output: StratifiedKFold(n_splits=5, random_state=27, shuffle=True)
```

LOOCV (very small datasets)

Leave-One-Out Cross-Validation (LOOCV) is the extreme case where each observation is its own validation fold. LOOCV is rarely used for modern datasets because it is computationally expensive and can have high variance, but it can be helpful for very small datasets (for example, fewer than 50 observations).

```
from sklearn.model_selection import LeaveOneOut

loocv = LeaveOneOut()

# Demonstration pattern only: do not use LOOCV on large datasets.
# Example: you could run LOOCV on a tiny sample for illustration.
```

Repeated K-Fold

Repeated K-Fold repeats K-Fold cross-validation multiple times with different random splits. This often produces a more stable estimate than a single K-Fold run, especially for small datasets.

```
from sklearn.model_selection import RepeatedStratifiedKFold

rskf = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=SEED)
rskf
```

Group K-Fold (non-independent samples)

Group K-Fold is used when samples are not independent. The key idea is that all records from the same entity (for example, the same customer) must stay together in either the training or validation fold. This prevents a subtle form of leakage where the model sees “almost the same” entity in both training and validation.

If your dataset contains a grouping identifier (such as a customer ID), you can pass a *groups* vector into cross-validation. If you do not have a true ID, do not invent one for real modeling; in that case, Group K-Fold is a concept to understand for when you do have grouped data.

```
from sklearn.model_selection import GroupKFold

# Example: choose a grouping column if it exists (adjust to your dataset)
group_col_candidates = [&quot;member_id&quot;, &quot;customer_id&quot;,
&quot;addr_state&quot;, &quot;zip_code&quot;]
group_col = next((c for c in group_col_candidates if c in X_train.columns), None)
gkf = GroupKFold(n_splits=5)

# Usage pattern:
# if group_col is not None:
#   groups = X_train[group_col].values
#   ... pass groups=groups into cross_validate / cross_val_score ...
```

TimeSeriesSplit (brief intro; expanded later)

Time series evaluation has a different rule: you cannot train on future data to predict the past. *TimeSeriesSplit* creates folds that respect ordering. This chapter introduces the concept briefly; a later time-series chapter expands it with forecasting-specific validation strategies.

In this Lending Club dataset, *issue_age_days* provides a simple time ordering proxy (smaller values are more recent). If this column exists, you can sort by it and demonstrate a time-aware split.

```
from sklearn.model_selection import TimeSeriesSplit

tss = TimeSeriesSplit(n_splits=5)

# Demonstration pattern (only if issue_age_days exists):
# X_train_sorted = X_train.sort_values(&quot;issue_age_days&quot;)
# y_train_sorted = y_train[np.argsort(X_train[&quot;issue_age_days&quot;].values)]
# Then use tss on the sorted arrays.
```

Pipelines and leakage

A critical professional habit is to place all preprocessing steps **inside** the pipeline. If you scale or encode the full dataset before cross-validation, you leak information from the validation folds into the training process, producing overly optimistic results.

The following preprocessing pattern (ColumnTransformer) is reusable throughout the chapter. It automatically handles numeric scaling and categorical one-hot encoding. The resulting *preprocessor* object becomes a building block for every model pipeline.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.base import BaseEstimator, TransformerMixin

# Optional utility transformer: ensure float64 output (helps with some estimators)

class EnsureFloat64(BaseEstimator, TransformerMixin):

  def fit(self, X, y=None):
    return self

  def transform(self, X):
    return X.astype(np.float64, copy=False)

numeric_cols = X_train.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = X_train.select_dtypes(exclude=[np.number]).columns.tolist()

numeric_pipe = Pipeline(steps=[
  (&quot;impute&quot;, SimpleImputer(strategy=&quot;median&quot;)),
  (&quot;scale&quot;, StandardScaler())
])

categorical_pipe = Pipeline(steps=[
  (&quot;impute&quot;, SimpleImputer(strategy=&quot;most_frequent&quot;)),
  (&quot;onehot&quot;, OneHotEncoder(handle_unknown=&quot;ignore&quot;,
sparse_output=False))
])

preprocessor = ColumnTransformer(
  transformers=[
    (&quot;num&quot;, numeric_pipe, numeric_cols),
    (&quot;cat&quot;, categorical_pipe, categorical_cols)
  ],
  remainder=&quot;drop&quot;
```

```
)

preprocessor
```

Now that we've setup the various cross-validation models, let's apply one of them (stratified k-fold) to the Lending Club dataset to see it in practice.

15.3CV in Practice

With a reusable preprocessor, you can define a baseline model pipeline once and then evaluate it with different cross-validation strategies. Logistic regression is a strong baseline classifier because it is fast, interpretable, and provides probabilities.

```
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(max_iter=2000, random_state=SEED)

model_lr = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;float&quot;, EnsureFloat64()),
  (&quot;lr&quot;, lr)
])

model_lr
```

Use *cross_val_score* for a single metric. For classification, *StratifiedKFold* is usually the default choice.

```
from sklearn.model_selection import cross_val_score

scores_acc = cross_val_score(
  model_lr,
  X_train,
  y_train,
  cv=skf,
  scoring=&quot;accuracy&quot;,
  n_jobs=-1
)

scores_acc.mean(), scores_acc.std()
```

```
# Output:
# (np.float64(0.9532219570405729), np.float64(0.004328963519622192))
```

Multiple metrics with cross_validate

In practice, teams rarely evaluate a model using only a single metric. Accuracy alone can be misleading when classes are imbalanced, and probability-based decisions depend not just on ranking ability but on the quality of predicted probabilities. The *cross_validate* function allows you to compute multiple evaluation metrics in a single cross-validation run, producing a richer and more realistic picture of model performance.

In the example below, the model is evaluated using six complementary metrics: overall accuracy, balanced accuracy, F1 score, ROC AUC, average precision, and log loss. For each metric, cross-validation produces a distribution of scores across folds rather than a single number. Reporting both the mean and the standard deviation helps quantify not only typical performance, but also how stable that performance is across different splits of the data.

```
from sklearn.model_selection import cross_validate

scoring = {
  &quot;accuracy&quot;: &quot;accuracy&quot;,
  &quot;balanced_accuracy&quot;: &quot;balanced_accuracy&quot;,
  &quot;f1&quot;: &quot;f1&quot;,
  &quot;roc_auc&quot;: &quot;roc_auc&quot;,
  &quot;avg_precision&quot;: &quot;average_precision&quot;,
  &quot;neg_log_loss&quot;: &quot;neg_log_loss&quot;
}

cv_results = cross_validate(
  model_lr,
  X_train,
  y_train,
  cv=skf,
  scoring=scoring,
  return_train_score=False,
```

```
        n_jobs=-1
      )

      {key: (np.mean(val), np.std(val)) for key, val in cv_results.items() if
key.startswith(&quot;test_&quot;)}


      # Output:
      # {'test_accuracy': (np.float64(0.9532219570405729),
      #   np.float64(0.004328963519622192)),
      #  'test_balanced_accuracy': (np.float64(0.7869637181142097),
      #   np.float64(0.016574219737666755)),
      #  'test_f1': (np.float64(0.9747524966678727),
      #   np.float64(0.0023343676756992743)),
      #  'test_roc_auc': (np.float64(0.9556364450780277),
      #   np.float64(0.006660690644702344)),
      #  'test_avg_precision': (np.float64(0.9951816277975277),
      #   np.float64(0.0010208160410535444)),
      #  'test_neg_log_loss': (np.float64(-0.1263226807833116),
      #   np.float64(0.0066620208002563155))}
```

These results illustrate why multiple metrics matter. Although overall accuracy is very high, the noticeably lower balanced accuracy reveals that performance differs between classes, a common issue in imbalanced datasets such as loan default prediction. The strong F1 score and average precision indicate good identification of the positive class, while the ROC AUC confirms solid ranking ability across thresholds.

The relatively small standard deviations across folds suggest that the model's performance is stable and not overly sensitive to how the data is split. This stability is one of the key benefits of cross-validation compared to a single train/test split, which can produce deceptively optimistic or pessimistic results depending on the chosen split.

Finally, note that scikit-learn reports loss-based metrics such as log loss using a negative sign (for example, *neg_log_loss*) so that higher values always indicate better performance within its scoring framework. When reporting results, multiply this value by -1 to return to the conventional interpretation where lower log loss indicates better calibrated probability estimates.

```
test_neg_ll = cv_results[&quot;test_neg_log_loss&quot;]
test_log_loss = -test_neg_ll
test_log_loss.mean(), test_log_loss.std()


# Output:
# (np.float64(0.1263226807833116), np.float64(0.0066620208002563155))
```

Because scikit-learn reports log loss as a negative value during cross-validation, it is common practice to convert *neg_log_loss* back to its conventional form by multiplying by -1. This makes the metric easier to interpret, since lower log loss corresponds to better-calibrated probability estimates.

In this case, the mean log loss across cross-validation folds is approximately 0.126, with a standard deviation of about 0.0067. The low average value indicates that the model's predicted probabilities are close to the true outcomes, not just in ranking observations correctly, but in assigning well-calibrated confidence levels.

The small standard deviation is equally important: it shows that probability quality is consistent across different folds of the training data. This suggests the model's probability estimates are stable and not overly sensitive to how the data is split, which is critical when predicted probabilities will be used for downstream decisions such as risk thresholds, pricing, or intervention strategies.

Log loss is especially valuable in applications like credit risk, fraud detection, and churn prediction, where decisions are rarely binary. Instead of asking only "which class is more likely," practitioners often need to ask "how confident should we be?" Cross-validation combined with log loss provides a principled way to answer that question before the model is ever deployed.

Choosing scoring metrics

Choosing a scoring metric is not just a technical detail; it expresses what you value. If you care most about ranking or probability thresholds, probability quality metrics (log loss, PR AUC) are often more aligned with decision-making than accuracy.

- **Regression scoring:** common options include *neg_mean_squared_error*, *neg_mean_absolute_error*, and *r2*.
- **Classification scoring (balanced):** *accuracy*, *f1*, *roc_auc*, *precision*, and *recall* are common starting points.
- **Classification scoring (imbalanced):** *balanced_accuracy*, *f1_macro*, and *average_precision* (PR AUC) often provide a more honest view than accuracy when positives are rare.
- **Probability-based decisions and log loss:** if you will use predicted probabilities for ranking, risk scores, or thresholds, include *neg_log_loss* in evaluation and tuning because it directly measures probability quality.
- **Scoring gotchas and make_scorer:** some metrics require probability outputs; some require choosing an averaging method; and some require custom scoring functions aligned to business costs.

Understanding why accuracy can mislead is essential. Consider the simplest possible model: one that predicts the majority class for every observation. In a dataset where 98% of cases belong to the negative class, this trivial model achieves 98% accuracy without learning anything at all. Its accuracy simply mirrors the class distribution. Metrics like balanced accuracy, macro-averaged F1, and recall per class expose this illusion because they evaluate whether the model is actually distinguishing between classes rather than riding the majority.

A common scoring gotcha is that log loss requires probabilities. If a model does not support *predict_proba*, you cannot compute log loss directly without

using probability calibration techniques (covered later). Another common gotcha is that a business metric (cost, profit, expected loss) is often not available as a built-in scorer.

The pattern below shows how to create a custom scorer with *make_scorer*. This example uses a simple “cost” idea for false negatives and false positives. The numbers (fp_cost=1.0 and fn_cost=5.0) are placeholders; in a real business, you would derive them from domain costs.

```
from sklearn.metrics import confusion_matrix, make_scorer

def expected_cost(y_true, y_pred, fp_cost=1.0, fn_cost=5.0):
  tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()

  return (fp_cost * fp) + (fn_cost * fn)

# Lower cost is better, so set greater_is_better=False
cost_scorer = make_scorer(expected_cost, greater_is_better=False)

scores_cost = cross_val_score(
  model_lr,
  X_train,
  y_train,
  cv=skf,
  scoring=cost_scorer,
  n_jobs=-1
)

scores_cost.mean(), scores_cost.std()


# Output:
# (np.float64(-77.2), np.float64(11.22319027727856))
```

The custom cost scorer returns a negative value because *make_scorer* was configured with *greater_is_better=False*. Internally, scikit-learn negates the returned cost so that “higher is better” remains consistent with its optimization framework. When interpreting results, focus on the magnitude rather than the sign: smaller absolute values indicate lower expected cost.

In this example, the mean cross-validated score corresponds to an average expected cost of about 77 units per fold, with a standard deviation of roughly

11. This tells us two important things. First, the model makes a non-trivial number of costly errors under the assumed cost structure, driven primarily by false negatives, which were assigned a higher penalty. Second, the variability across folds indicates that cost-sensitive performance is somewhat sensitive to how the data is split.

Unlike accuracy or F1 score, this metric directly encodes business priorities. By weighting false negatives five times more than false positives, the evaluation emphasizes avoiding missed defaults rather than maximizing overall correctness. This is often a better reflection of real-world objectives, where different mistakes carry very different consequences.

The key takeaway is that model selection should follow the decision context. A model with slightly lower accuracy but substantially lower expected cost may be the better choice in production. Cross-validation combined with custom scorers allows teams to make those trade-offs explicit, measurable, and repeatable.

15.4Learning curves

Once you have a reliable evaluation setup using cross-validation, the next question is *why* a model performs the way it does. Learning curves answer this by showing how training and validation performance change as the size of the training data increases. They help you diagnose whether a model is suffering from bias, variance, or neither.

- What learning curves show: training score and cross-validated validation score plotted against increasing training set size.
- Bias vs variance patterns: whether errors come from an overly simple model or an overly complex one.

- CV-based curves in sklearn: learning curves are computed using cross-validation internally, not a single split.
- Practical decisions: whether to collect more data, increase model complexity, or simplify the model.

Learning curves plot two lines. The training curve shows how well the model fits the data it was trained on, while the validation curve shows how well it generalizes to unseen data. The gap between these curves—and whether they converge—provides immediate insight into model behavior.

A high-bias (underfitting) pattern appears when both training and validation scores are low and converge quickly. This indicates that the model is too simple to capture the underlying structure of the data. Common remedies include adding features, increasing model complexity, or reducing regularization.

It is worth distinguishing a high-bias pattern from a simpler issue: optimization convergence. If the learning algorithm itself has not finished converging—for example, because *max_iter* is set too low—you may see low or erratic scores that improve when you increase the iteration count. That is an optimization problem, not a capacity problem. By contrast, when a high-bias learning curve shows both scores plateauing at a modest level, the model has converged but simply cannot represent the underlying pattern. In that case, increasing *max_iter* will not help; you need a more flexible model or richer features.

A high-variance (overfitting) pattern appears when the training score is high but the validation score is much lower, with a persistent gap even as training size increases. This indicates that the model memorizes training data but fails to generalize. Remedies include collecting more data, simplifying the model,

increasing regularization, or performing feature selection (covered in the next chapter).

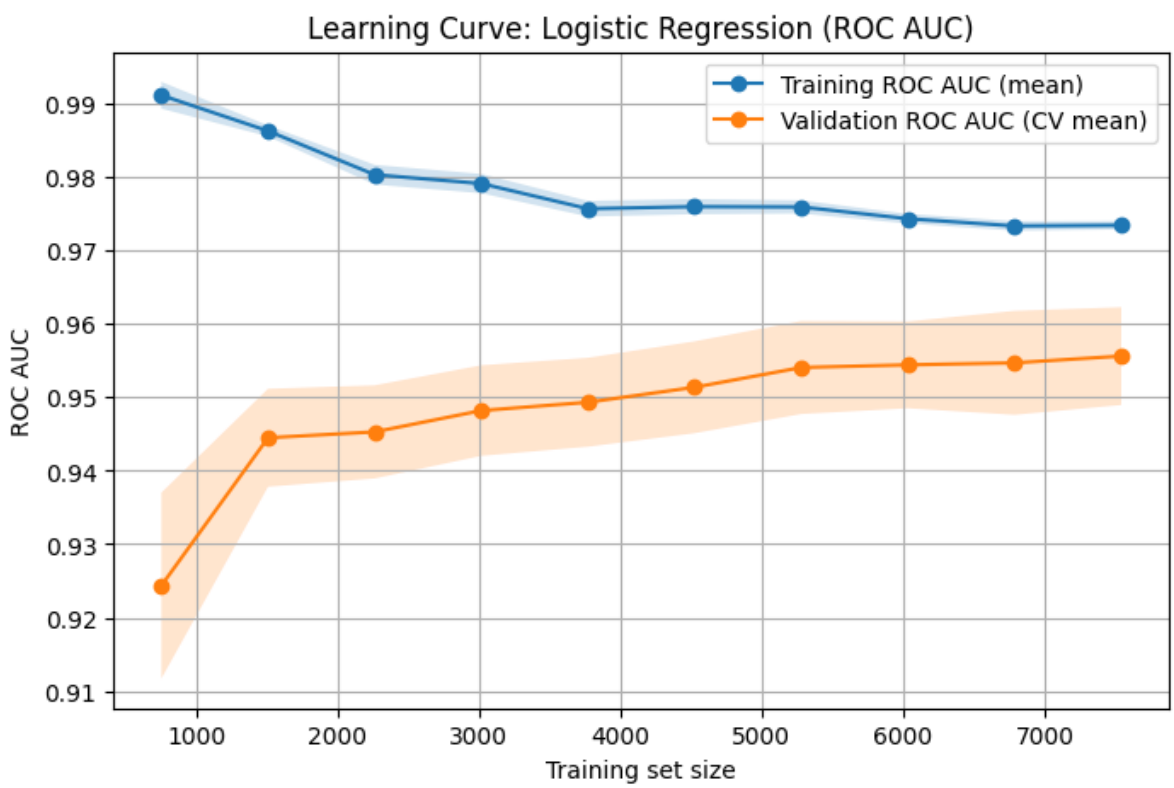
A well-fit model shows both training and validation scores increasing and converging to a high value. In this case, additional data may yield diminishing returns, and further gains may require better features or a different model class.

Importantly, learning curves in scikit-learn are always computed using cross-validation. You should avoid generating learning curves from a single validation split, as those curves are noisy and can lead to incorrect conclusions.

```
from sklearn.model_selection import learning_curve
import numpy as np

train_sizes, train_scores, val_scores = learning_curve(
  model_lr,
  X_train,
  y_train,
  cv=skf,
  scoring=&quot;roc_auc&quot;,
  train_sizes=np.linspace(0.1, 1.0, 10),
  n_jobs=-1
)

train_mean = train_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
val_mean = val_scores.mean(axis=1)
val_std = val_scores.std(axis=1)
train_sizes, train_mean, val_mean


# Output:
# (array([ 754, 1508, 2262, 3016, 3771, 4525, 5279, 6033, 6787, 7542]),
#  array([0.9911892 , 0.98632185, 0.98036249, 0.97915087, 0.97572068,

#         0.97600776, 0.97596276, 0.97435747, 0.97337691, 0.97346763]),
#  array([0.9243653 , 0.94450434, 0.94530303, 0.94820811, 0.94934411,

#         0.95138631, 0.95407962, 0.95444045, 0.95471374, 0.95563645]))
```

Now that we have the values we need, let's plot them for visual interpretation:

```
import matplotlib.pyplot as plt

# You already computed these:
# train_sizes, train_mean, train_std, val_mean, val_std
plt.figure(figsize=(8, 5))
plt.plot(train_sizes, train_mean, marker=&quot;o&quot;, label=&quot;Training ROC AUC
(mean)&quot;)
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std,
alpha=0.2)
plt.plot(train_sizes, val_mean, marker=&quot;o&quot;, label=&quot;Validation ROC AUC
(CV mean)&quot;)
plt.fill_between(train_sizes, val_mean - val_std, val_mean + val_std, alpha=0.2)
plt.title(&quot;Learning Curve: Logistic Regression (ROC AUC)&quot;)
plt.xlabel(&quot;Training set size&quot;)
plt.ylabel(&quot;ROC AUC&quot;)
plt.legend()
plt.grid(True)
plt.show()
```



The learning curve plot shows how model performance changes as the amount of training data increases, using cross-validation to estimate both training and validation performance. The training curve reflects how well the model fits

the data it sees, while the validation curve reflects how well it generalizes to unseen data.

In this example, the training ROC AUC starts extremely high (just above 0.99) when the model is trained on a small subset of the data. This indicates that with limited data, the logistic regression model can fit the training samples almost perfectly.

As the training set size increases, the training ROC AUC gradually declines and stabilizes around 0.973–0.974. This pattern is expected: as the model sees more diverse data, it becomes harder to fit every observation perfectly, and the training performance settles at a more realistic level.

The validation ROC AUC increases steadily as more training data is added, rising from roughly 0.92 to about 0.956. This upward trend indicates that the model continues to benefit from additional data and that generalization improves as the training set grows.

The gap between the training and validation curves is largest at small training sizes and narrows as more data is added. This shrinking gap signals that early overfitting pressure is being reduced by additional data rather than by model changes.

Importantly, the validation curve has not fully flattened at the right edge of the plot. This suggests that while gains are becoming smaller, additional data could still yield modest improvements in performance.

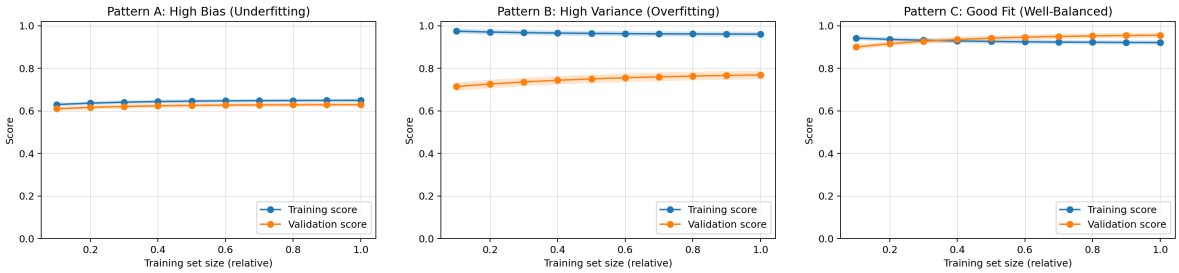
From a bias–variance perspective, this pattern reflects a model with moderate variance rather than high bias. The model is expressive enough to learn useful structure, but its generalization improves primarily through more data rather than increased complexity.

Practically, this learning curve supports three conclusions: the model is not underfitting, the model is not severely overfitting, and collecting more data is likely to help more than increasing model complexity at this stage.

In a real project, this diagnosis would justify proceeding to hyperparameter tuning or feature selection only after data availability has been considered, rather than immediately switching to a more complex algorithm.

Common Patterns and Their Interpretation

It would probably help to see a few more examples of learning curves to get a feel for how you should interpret results and make decisions. Examine the charts below and read the explanations in the table below.



Three common learning-curve patterns and what they typically imply about model bias, variance, and next steps.

Table 15.1
Interpreting common learning-curve patterns

Pattern	What you see	What it usually means	Common decisions and next steps

Pattern	What you see	What it usually means	Common decisions and next steps
Pattern A: High bias (underfitting)	Training and validation scores are both relatively low and close together, and they converge quickly as training size grows.	The model is too simple for the signal in the data, or the feature set is not expressive enough; adding more data alone usually will not help much.	• Increase model capacity (for example, add interactions, use a nonlinear model, or engineer richer features). • Reduce regularization if using a strongly regularized model. • Add better predictors rather than collecting more rows. • Check label noise and ceiling effects, which can limit achievable performance.
Pattern B:			

Pattern	What you see	What it usually means	Common decisions and next steps
High variance (overfitting)	Training score is high while validation score is much lower, and the gap remains large or shrinks only slowly as training size increases.	The model is fitting idiosyncrasies of the training data that do not generalize well; variance dominates model error.	• Collect more training data if feasible. • Simplify the model (shallower trees, fewer parameters, smaller networks). • Increase regularization strength. • Apply feature selection or remove noisy features (expanded in the next chapter). • Verify preprocessing and cross-validation design to rule out data leakage.

Pattern	What you see	What it usually means	Common decisions and next steps
Pattern C: Good fit (well-balanced)	Both training and validation scores are high, the gap is small, and the curves begin to plateau as training size increases.	The model family is well matched to the problem; additional complexity or data may yield diminishing returns.	• Focus on hyperparameter tuning rather than changing model families. • Evaluate whether the chosen metric aligns with business objectives. • Improve probability quality and decision thresholds if using risk scores. • Shift attention to robustness, subgroup performance, and monitoring plans.

15.5Validation curves

Before diving into validation curves, it is worth recalling what a *hyperparameter* is. A hyperparameter is a model setting chosen *before training* that controls how the learning algorithm behaves, rather than a parameter learned directly from the data.

You have already worked with hyperparameters earlier in the book. For example, when tuning decision trees, you adjusted settings such as *max_depth* and *min_samples_leaf* to control how complex the tree could become. In logistic regression, you adjusted regularization strength using the *C* parameter.

Validation curves focus on *one hyperparameter at a time*. They help you see how sensitive model performance is to that choice and whether increasing or decreasing complexity improves generalization.

While learning curves diagnose problems related to data size and overall model capacity, validation curves focus on *a single hyperparameter*. They show how training and validation performance change as that hyperparameter varies, holding everything else constant.

- Single-hyperparameter sensitivity: how performance responds as one hyperparameter increases or decreases.
- Bias/variance interpretation: low values often correspond to high bias, high values to high variance.
- CV-based validation_curve in sklearn: scores are computed using cross-validation, not a single split.
- Limitations: only one hyperparameter at a time, so interactions are not captured.

Validation curves typically show a U-shaped pattern for the validation score. On the left, the model is too simple and underfits. On the right, the model is too complex and overfits. The optimal hyperparameter value lies near the peak of the validation curve.

Like learning curves, validation curves in scikit-learn use cross-validation internally. This ensures that observed patterns reflect generalization behavior rather than noise from a single train/validation split.

```
from sklearn.model_selection import validation_curve
import numpy as np

param_range = np.logspace(-3, 3, 7)

train_scores, val_scores = validation_curve(
  model_lr,
  X_train,
  y_train,
  param_name=&quot;lr__C&quot;,
  param_range=param_range,
  cv=skf,
  scoring=&quot;roc_auc&quot;,
  n_jobs=-1
)

train_mean = train_scores.mean(axis=1)
val_mean = val_scores.mean(axis=1)
param_range, train_mean, val_mean


# Output:
# (array([1.e-03, 1.e-02, 1.e-01, 1.e+00, 1.e+01, 1.e+02, 1.e+03]),
#  array([0.89421271, 0.94402687, 0.95984995, 0.97346763, 0.98180775,

#         0.98468622, 0.98596522]),
#  array([0.89021749, 0.93978624, 0.95206725, 0.95563645, 0.94854225,

#         0.93437912, 0.91446818]))
```

The code above computes a validation curve for logistic regression by systematically varying the *C* hyperparameter and measuring performance on both training folds and validation folds. The goal is to see how model complexity (as controlled by regularization) changes performance and to identify a “sweet spot” where the model generalizes well.

- **model_lr** is the reusable pipeline you built earlier. It includes preprocessing steps (such as imputation and one-hot encoding) and ends with a logistic regression model. Because it is a pipeline, hyperparameters inside the model are referenced using the step name prefix (for example, *lr__C*).
- **X_train, y_train** are the training-set features and labels. The validation curve is computed using only the training set so that the final test set remains untouched for the end-of-chapter evaluation workflow.
- **param_** specifies which hyperparameter to vary. Here, *C* is the inverse regularization strength for logistic regression. The *lr__* prefix targets the logistic regression step inside the pipeline.
- **param_range=np.logspace(-3, 3, 7)** defines the values of *C* to test. Using a log scale is standard for regularization because the effect of *C* changes dramatically across orders of magnitude (for example, 0.001 vs 0.01 vs 0.1).
- **cv=skf** sets the cross-validation strategy (Stratified K-Fold). Stratification preserves the class balance in each fold, making the validation estimates more reliable for classification problems with uneven class frequencies.
- **scoring="roc_auc"** chooses the evaluation metric. ROC AUC measures how well the model ranks positives above negatives across all possible thresholds, which is often more informative than accuracy in credit-risk and default prediction settings.
- **n_jobs=-1** parallelizes the computation across CPU cores to reduce runtime because each point in *param_range* requires multiple model fits across CV folds.

Regularization is a technique used to control model complexity by discouraging extreme or unstable coefficient values. In logistic regression, regularization adds a penalty to the loss function that increases as the model’s

coefficients grow larger. This pushes the optimizer toward smaller coefficients, which usually produces a simpler decision boundary that generalizes better to new data.

Without regularization, logistic regression will choose coefficient values that best fit the training data, even if that fit relies on patterns that do not persist outside the sample. This can increase overfitting: training performance looks excellent, but validation performance suffers because the model is overly sensitive to noise and small quirks in the training set.

The *C* parameter controls how strongly regularization is applied. Because *C* is defined as the inverse of regularization strength, smaller values of *C* impose stronger regularization (more coefficient shrinkage), while larger values of *C* impose weaker regularization (less shrinkage and more flexibility).

This creates a classic bias–variance trade-off. With very small *C* (strong regularization), the model may underfit because it is too constrained to capture the signal in the data. With very large *C* (weak regularization), the model may overfit because it can fit training noise too easily. The validation curve helps you see where your model sits on this spectrum.

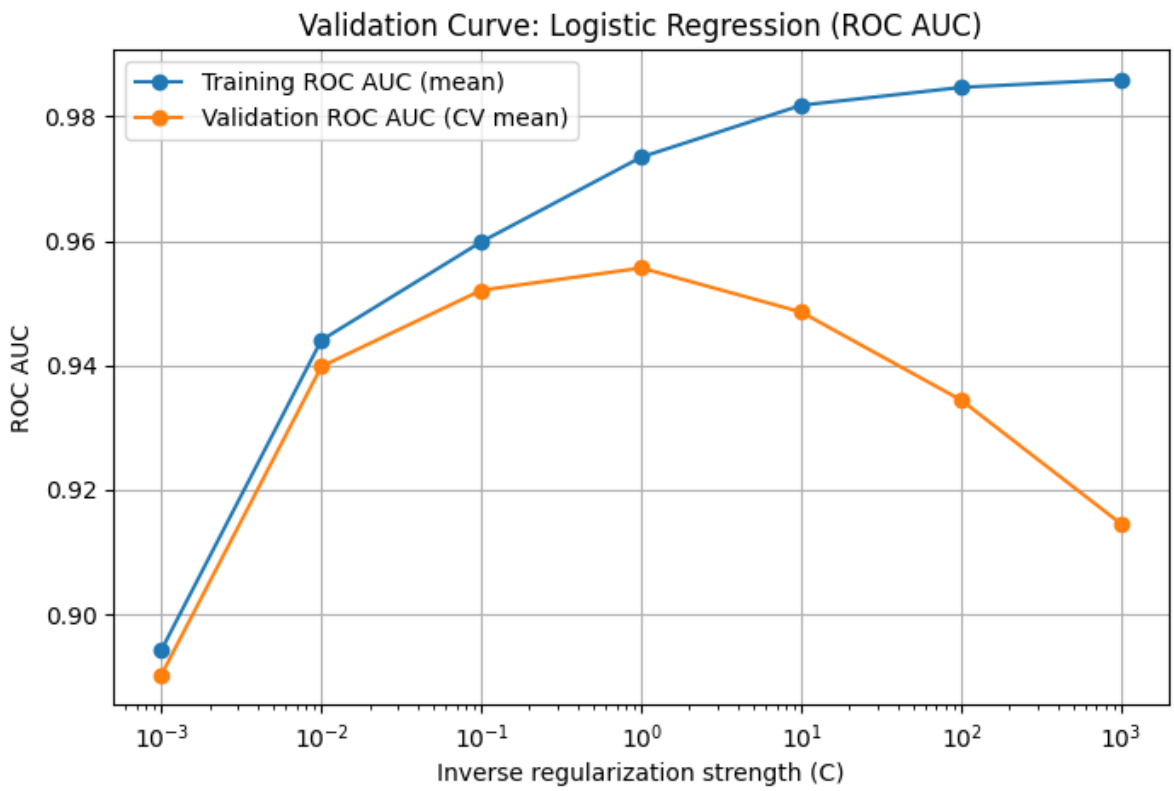
Interpreting the curve usually follows a consistent pattern: as *C* increases, training performance tends to improve because the model becomes more flexible. Validation performance often improves at first (the model is no longer underfitting), then peaks, and eventually declines if the model becomes too flexible and starts overfitting. The best *C* values are typically near the peak of the validation curve, where validation performance is highest and the gap between training and validation remains relatively small.

At this stage, you are using validation curves as a diagnostic tool: you are not doing an exhaustive search across many interacting hyperparameters. Instead,

you are building intuition about how one key hyperparameter influences generalization. Once you see where the “good region” is for *C*, you can narrow your tuning ranges and move on to more systematic search methods in the next pass of the chapter.

Let's plot the results again:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

plt.plot(
  param_range,
  train_mean,
  marker=&quot;o&quot;,
  label=&quot;Training ROC AUC (mean)&quot;
)

plt.plot(
  param_range,
  val_mean,
  marker=&quot;o&quot;,
  label=&quot;Validation ROC AUC (CV mean)&quot;
)

plt.xscale(&quot;log&quot;)
plt.xlabel(&quot;Inverse regularization strength (C)&quot;)
plt.ylabel(&quot;ROC AUC&quot;)
plt.title(&quot;Validation Curve: Logistic Regression (ROC AUC)&quot;)
plt.legend()
plt.grid(True)
plt.show()
```



The validation curve shows how model performance changes as the inverse regularization strength (C) varies. In logistic regression, smaller values of C correspond to stronger regularization and simpler models, while larger values of C correspond to weaker regularization and more complex models.

On the left side of the curve (very small C values), both the training and validation ROC AUC scores are relatively low and close together. This pattern indicates high bias, meaning the model is underfitting because regularization is too strong to capture the underlying signal in the data. As C increases in this region, both scores improve, showing that the model benefits from additional flexibility.

In the middle of the curve, the validation ROC AUC reaches its maximum value (around $C \approx 1$). At this point, the model achieves its best balance between bias and variance. Training performance is high, validation

performance is maximized, and the gap between the two remains relatively small. This region represents the best expected generalization performance and is the most appropriate choice for deployment or further evaluation.

On the right side of the curve (large *C* values), the training ROC AUC continues to increase while the validation ROC AUC begins to decline. This divergence indicates high variance, or overfitting. The model is fitting noise in the training data that does not generalize well to unseen data, as evidenced by the widening gap between training and validation scores.

The key lesson from this validation curve is that hyperparameter selection should be based on validation performance rather than training performance. Validation curves make sensitivity to model complexity visible and motivate systematic tuning methods, such as grid search, which build directly on this idea.

Validation Curves for Decision Trees

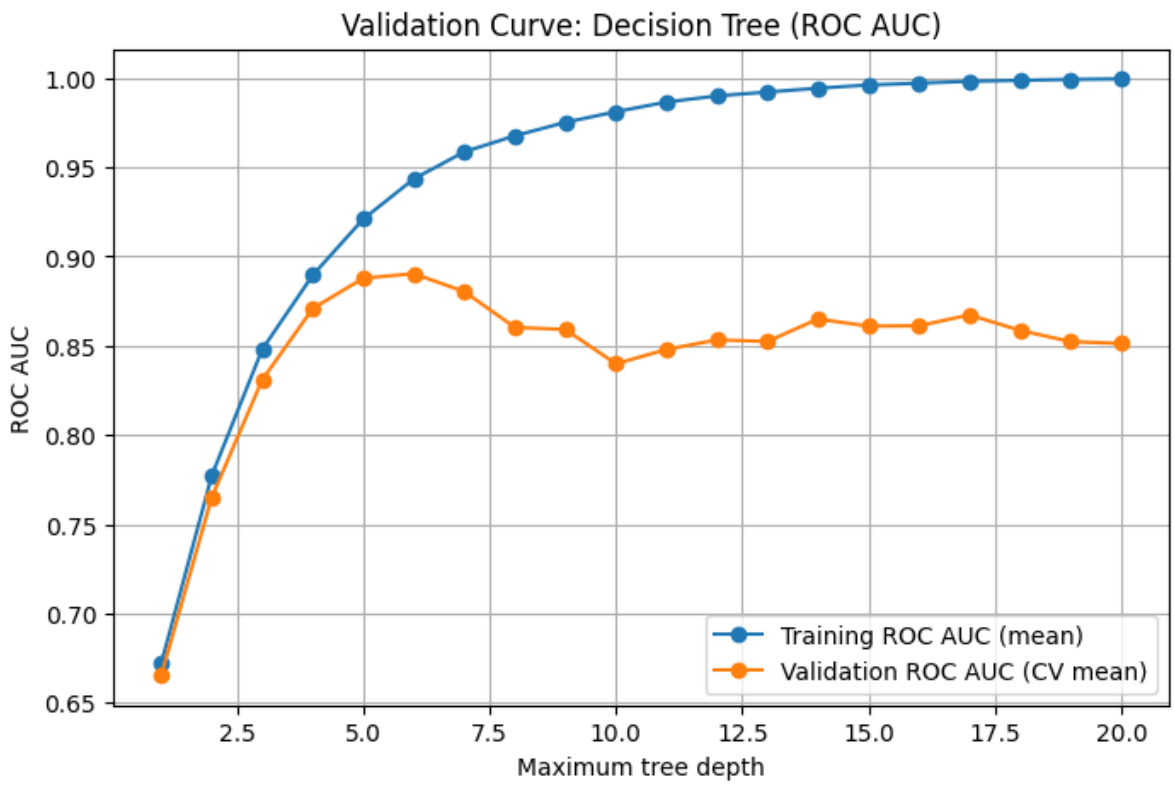
To reinforce the idea that validation curves apply to many model families, the example below shows a validation curve for a decision tree classifier. Decision trees expose hyperparameters that control structure directly, making them especially intuitive for understanding bias–variance trade-offs.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import validation_curve
import numpy as np

param_range = np.arange(1, 21)

model_tree = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),   # reuse the exact preprocessor you built
earlier
  (&quot;tree&quot;, DecisionTreeClassifier(random_state=SEED))
])

train_scores, val_scores = validation_curve(
```

```
  model_tree,
  X_train,
  y_train,
  param_name=&quot;tree__max_depth&quot;,   # pipeline step name prefix
  param_range=param_range,
  cv=skf,
  scoring=&quot;roc_auc&quot;,
  n_jobs=-1,
  error_score=&quot;raise&quot;             # keep this while developing
)

train_mean = train_scores.mean(axis=1)
val_mean = val_scores.mean(axis=1)
param_range, train_mean, val_mean
```

The code above computes a validation curve for a decision tree classifier by systematically varying the *max_depth* hyperparameter and measuring performance on both training folds and validation folds. The goal is to see how model complexity (as controlled by the maximum depth of the tree) changes performance and to identify a “sweet spot” where the model generalizes well. Now let's plot the results again:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

plt.plot(
  param_range,
  train_mean,
  marker=&quot;o&quot;,
  label=&quot;Training ROC AUC (mean)&quot;
)

plt.plot(
  param_range,
  val_mean,
  marker=&quot;o&quot;,
  label=&quot;Validation ROC AUC (CV mean)&quot;
)

plt.xlabel(&quot;Maximum tree depth&quot;)
plt.ylabel(&quot;ROC AUC&quot;)
plt.title(&quot;Validation Curve: Decision Tree (ROC AUC)&quot;)
plt.legend()
plt.grid(True)
plt.show()
```



The validation curve above shows how a decision tree's performance changes as you increase *max_depth*, which directly controls model complexity. A shallow tree (low max_depth) can only learn very simple decision rules, while a deeper tree can create many more splits and capture increasingly specific patterns in the training data.

On the left side of the plot (depth 1–3), both the training and validation ROC AUC values are relatively low and close together. This is a high-bias pattern: the model is too constrained to capture the signal in the data, so it underfits. As max_depth increases from about 2 to 5, both curves rise sharply, which tells you that the model benefits from additional complexity.

Around depths 5–6, the validation ROC AUC reaches its highest point (roughly 0.89). This region is the best trade-off between bias and variance: the tree is complex enough to learn meaningful structure, but not so complex that

it starts memorizing noise. Notice that the training ROC AUC is already quite high by this point, but the key decision should be guided by the *validation* curve, not the training curve.

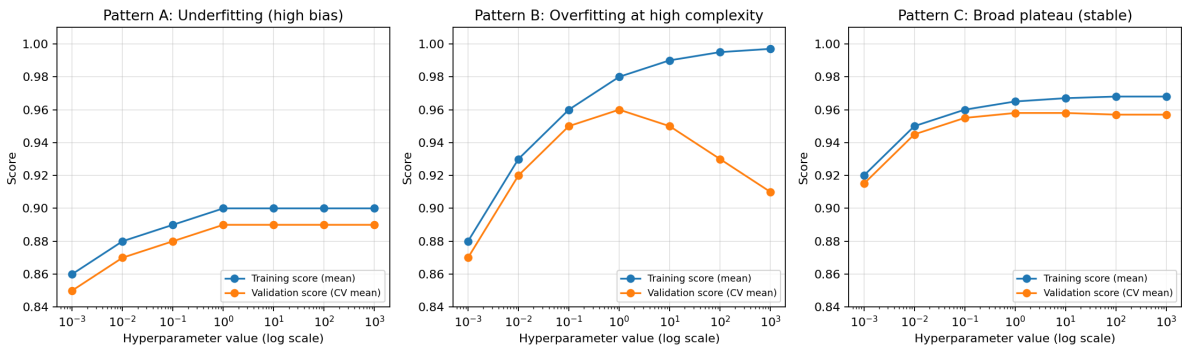
After about depth 6, the training ROC AUC continues to climb steadily toward 1.00, but the validation ROC AUC begins to decline and then flattens at a lower level (roughly mid-0.85s). This widening gap is the classic high-variance pattern: deeper trees are fitting quirks of the training folds that do not generalize well to unseen data. In other words, the model becomes increasingly overfit as depth increases.

A reasonable decision from this curve is to choose *max_depth* near the validation peak (around 5 or 6). In practice, teams often prefer the *simplest* setting that achieves near-best validation performance, because simpler models tend to be more stable and easier to explain. For example, if depth 5 performs almost as well as depth 6, depth 5 is often the better choice.

This plot also reinforces an important lesson: a model can keep improving on training data while getting worse on validation data. That is why validation curves are valuable—they make overfitting visible. Once you identify the "good region" (here, roughly depths 4–7), you can narrow your hyperparameter ranges and then use more systematic tuning methods (such as grid search or randomized search) to finalize your configuration.

Common Patterns and Their Interpretation

It can help to see a few generic validation-curve patterns so you develop an intuition for what they imply about model complexity, bias, variance, and hyperparameter choices.



Three common validation-curve patterns and what they typically imply about hyperparameter choice, underfitting, overfitting, and next steps.

Alt text: Three side-by-side validation-curve charts on a log-scaled hyperparameter axis showing training score and cross-validated validation score: Pattern A has both curves low and close (underfitting), Pattern B has validation peaking then dropping while training keeps rising (overfitting at high complexity), and Pattern C shows both curves high with a broad flat plateau (stable performance across a wide range).

Table 15.2
Interpreting common validation-curve patterns

Pattern	What you see	What it usually means	Common decisions and next steps
Pattern A: Underfitting (high bias)	Training and validation scores are both relatively low, close together, and improve only modestly as the hyperparameter changes.	The model family or feature set is not expressive enough for the signal in the data; changing this single hyperparameter cannot unlock strong performance.	• Move to a more flexible model (for example, nonlinear or tree-based methods) or add richer features. • Reduce regularization if

Pattern	What you see	What it usually means	Common decisions and next steps
			the hyperparameter controls regularization strength and the model is overly constrained. • Try feature engineering before collecting more data, because more data alone often does not fix bias. • Check label quality and whether the chosen metric has a performance ceiling due to noise.
Pattern B: Overfitting	Training score steadily increases	The hyperparameter is pushing the	

Pattern	What you see	What it usually means	Common decisions and next steps
at high complexity	as complexity increases, but validation score rises only up to a point and then declines.	model into a high-variance regime; beyond the peak, the model fits training quirks that do not generalize.	• Choose the hyperparameter value near the validation peak (or the smallest value within one standard deviation of the best score). • Increase regularization or reduce model complexity to close the train-validation gap. • Use more data if feasible; variance problems often improve with additional training rows. • If the peak is noisy, use more folds or repeated cross-validation for a

Pattern	What you see	What it usually means	Common decisions and next steps
			more stable estimate.
Pattern C: Broad plateau (stable)	Validation score reaches a high value and stays relatively flat across a wide range of hyperparameter settings; training score is also high and close.	The model is not very sensitive to this hyperparameter within the tested range; many settings generalize similarly well.	• Prefer simpler or more regularized settings within the plateau to reduce overfitting risk and improve robustness. • Shift effort to other hyperparameters or move to a broader search (because single-parameter tuning is not the bottleneck). • Use secondary criteria (training time, inference speed, interpretability) to pick among

Pattern	What you see	What it usually means	Common decisions and next steps
			equivalent-performing settings. • Verify performance with your final held-out test set after selecting the configuration.

15.6Hyperparameter Tuning

Up to this point, you have used cross-validation, learning curves, and validation curves to *diagnose* model behavior. Hyperparameter tuning moves from diagnosis to *optimization*: systematically searching for the hyperparameter settings that produce the best generalization performance.

In professional machine learning workflows, hyperparameter tuning is never ad hoc. It is performed using structured search procedures that control overfitting, support reproducibility, and allow results to be defended to technical and non-technical stakeholders.

The problem with manual tuning

A common beginner approach is to manually adjust one hyperparameter at a time, re-run evaluation, and keep the best-looking result. While intuitive, this

approach has serious limitations.

- Hyperparameters interact in complex ways; tuning one at a time can miss strong combinations.
- Manual search explores only a tiny fraction of the possible configuration space.
- Repeated experimentation on the same validation folds increases the risk of overfitting to the validation process itself.
- Results are difficult to reproduce or justify without a formal search procedure.

Systematic search methods address these issues by applying cross-validation consistently across all candidate hyperparameter combinations.

GridSearchCV: Exhaustive Search

GridSearchCV performs an *exhaustive search* over all combinations of specified hyperparameter values. For each combination, the model is evaluated using cross-validation, and the average validation score is recorded.

Grid search is most appropriate when you have a small number of important hyperparameters and reasonably narrow ranges informed by prior knowledge or diagnostic tools such as validation curves.

Before you run a grid search, it helps to clarify which hyperparameters you are varying and what they control.

- **lr__C** is the inverse regularization strength for logistic regression. Smaller values apply stronger regularization (simpler model, higher bias). Larger values apply weaker regularization (more flexible model, higher variance).

- **lr__l1_ratio** controls the mix of L1 and L2 regularization. A value of *0.0* applies pure L2 (ridge-style), which shrinks all coefficients toward zero. A value of *1.0* applies pure L1 (lasso-style), which can shrink some coefficients to exactly zero—behaving like a light form of feature selection. Values between 0 and 1 produce elastic net, a blend of both.

The *l1_ratio* parameter is the standard way to control regularization type going forward. The older *penalty* parameter was deprecated in scikit-learn 1.8 and will be removed in 1.10. During the transition (scikit-learn < 1.10), you must also set *penalty='elasticnet'* for *l1_ratio* to take effect; once scikit-learn 1.10 is released, *penalty* can be removed entirely. The *saga* solver supports all values of *l1_ratio* (0.0 through 1.0) and works for both binary and multinomial classification, making it a good general-purpose choice.

```
from sklearn.model_selection import GridSearchCV

# Grid includes C (regularization strength) and l1_ratio (regularization type).
# l1_ratio=0.0 is pure L2 (ridge), l1_ratio=1.0 is pure L1 (lasso).
# penalty='elasticnet' needed for sklearn &lt;1.10; remove once sklearn ≥1.10
param_grid = {
  &quot;lr__C&quot;: [0.01, 0.1, 1.0, 10.0],
  &quot;lr__l1_ratio&quot;: [0.0, 1.0],
  &quot;lr__penalty&quot;: [&quot;elasticnet&quot;],
  &quot;lr__solver&quot;: [&quot;saga&quot;]
}

grid_search = GridSearchCV(
  estimator=model_lr,
  param_grid=param_grid,
  cv=skf,
  scoring=&quot;roc_auc&quot;,
  n_jobs=-1
)

grid_search.fit(X_train, y_train)
grid_search.best_params_, grid_search.best_score_

# Example output (yours may differ):
# ({'lr__C': 1.0, 'lr__l1_ratio': 0.0}, np.float64(0.9556364450780277))
```

Each point in the grid represents a full cross-validation run. If there are p hyperparameters with k values each, grid search evaluates k^p combinations.

This cost multiplies again by the number of cross-validation folds.

You may see a warning about “skipping features without any observed values” during imputation. This can occur inside cross-validation because each fold is fit on a subset of the training data. If a feature happens to be entirely missing within one fold’s training subset, the median imputer cannot compute a median for that fold and temporarily drops that feature for that fold. This warning is useful because it reveals that some columns can be extremely sparse or inconsistently populated.

If you see this warning repeatedly, it is often a sign that the feature should be handled differently (for example, treated as missingness-indicator-only, dropped entirely, or imputed with a different strategy). In many cases, however, the workflow still runs correctly and the warning simply reflects the realities of sparse columns when you evaluate models across many folds.

After fitting, GridSearchCV exposes several useful attributes that help you understand what it found and reuse the best configuration:

- **best_params_**: the hyperparameter combination with the highest mean cross-validated score for the scoring metric you chose (here, ROC AUC).
- **best_score_**: the corresponding mean validation score across folds for that best combination.
- **best_estimator_**: a fully trained pipeline configured with the best hyperparameters (useful for prediction and for final evaluation on the untouched test set).
- **cv_results_**: detailed scores, ranks, and timing information for all tested configurations (useful for diagnosing trade-offs and confirming that the “winner” is meaningfully better than alternatives).

The key idea is that GridSearchCV selects hyperparameters based on cross-validated validation performance, not training performance. That makes the selection process more reliable than manually trying a few settings and trusting a single split.

Also notice that the “best” hyperparameters are defined relative to your scoring metric. If you switch from ROC AUC to log loss or average precision, the best configuration can change because different metrics reward different kinds of model behavior.

Because grid search cost grows rapidly as you add hyperparameters and values, it becomes impractical for large search spaces. In those cases, randomized search provides a more efficient alternative by exploring a wider space with a fixed computational budget.

RandomizedSearchCV: Efficient Exploration

RandomizedSearchCV explores the hyperparameter space by randomly sampling combinations from specified distributions rather than evaluating every possible combination. This makes it possible to search much larger or continuous ranges of values while keeping the total computation under control.

The key difference from GridSearchCV is that you explicitly control the computational budget using *n_iter*. Instead of growing exponentially with the number of hyperparameters, the cost of randomized search is fixed and predictable.

Only a few parameters differ from grid search, but they change how the search behaves:

- **param_distributions** replaces *param_grid*. Instead of listing discrete values, you provide probability distributions to sample from. This allows randomized search to explore continuous ranges and orders of magnitude.
- **n_iter** controls how many random hyperparameter combinations are evaluated. Each iteration corresponds to one full cross-validation run. Increasing *n_iter* improves coverage of the search space at the cost of additional computation.
- **random_state** makes the random sampling reproducible so that results can be replicated and compared.

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform

# Log-uniform sampling of C from 0.001 to 1000 (appropriate for regularization
parameters)
param_dist = {
  &quot;lr__C&quot;: loguniform(1e-3, 1e3)
}

random_search = RandomizedSearchCV(
  estimator=model_lr,
  param_distributions=param_dist,
  n_iter=50,
  cv=skf,
  scoring=&quot;roc_auc&quot;,
  n_jobs=-1,
  random_state=SEED
)

random_search.fit(X_train, y_train)
random_search.best_params_, random_search.best_score_


# Output:
# /usr/local/lib/python3.12/dist-packages/sklearn/impute/_base.py:635: UserWarning:
# Skipping features without any observed values: ['term' 'emp_length_years'].
# ({'lr__C': np.float64(0.7236414654346375)}, np.float64(0.9555691263522705))
```

As with grid search, you may see warnings related to sparse features during cross-validation. These occur for the same reason: each fold is trained on a subset of the data, and some columns may be entirely missing within a

particular fold. The warning does not invalidate the results, but it does highlight features that may require special handling.

The output shows the sampled hyperparameter value that achieved the highest mean cross-validated ROC AUC. In this run, a *C* value of approximately 0.72 performed best, producing a validation score very close to the best score found by grid search.

This illustrates an important practical insight: randomized search often finds a near-optimal solution with far fewer evaluations than grid search. Even though it does not exhaustively test all combinations, it frequently reaches comparable performance at a fraction of the computational cost.

Unlike grid search, the computational cost of randomized search is fixed at *n_iter* × *CV folds*. This predictability makes randomized search the preferred choice when working with many hyperparameters, wide ranges, or limited time and computing resources.

Successive Halving (compute-aware search) <- Optional (won't be in the assignment)

As models and datasets grow larger, the dominant cost of hyperparameter tuning is not choosing values—it is *training time*. Exhaustive grid search and even randomized search spend the same amount of computation on clearly poor hyperparameter combinations as they do on promising ones. Successive halving addresses this inefficiency by allocating computational budget progressively.

The core idea behind successive halving is simple: start by evaluating *many hyperparameter configurations* using a small amount of training effort, then repeatedly eliminate the worst performers while increasing the training budget

for the remaining candidates. At each stage, only the most promising configurations survive to receive more resources.

In scikit-learn, this approach is implemented through *HalvingGridSearchCV* and *HalvingRandomSearchCV*. These estimators follow the same conceptual workflow as grid and random search, but add an additional dimension: how much data, time, or training iterations each configuration receives.

- **Early elimination:** Poor hyperparameter combinations are discarded quickly instead of being fully trained.
- **Progressive budgeting:** Promising configurations receive more data or more training iterations in later rounds.
- **Compute efficiency:** Total training time is often dramatically lower than exhaustive grid search.

Conceptually, successive halving mirrors how practitioners tune models in the real world. Early experiments are rough and inexpensive, used to rule out bad ideas quickly. Only a small number of candidates earn the right to be trained carefully and evaluated thoroughly.

Successive halving is most useful when training is expensive, datasets are large, or the hyperparameter space is wide. For small datasets or simple models, the added complexity may not provide meaningful benefits over randomized search.

Important: Successive halving does not replace the need for sound diagnostics. Learning curves and validation curves still play a critical role in understanding bias, variance, and reasonable hyperparameter ranges. Successive halving simply makes the search process more computationally efficient once those ranges are known.

For this book, focus on mastering *GridSearchCV* and *RandomizedSearchCV*. Successive halving is included to build awareness of modern, compute-aware optimization strategies that you will encounter in production systems and automated machine learning platforms.

Optional example: The code below demonstrates how to use successive halving with a logistic regression pipeline. The structure closely mirrors *GridSearchCV* and *RandomizedSearchCV*, but the search allocates computation progressively instead of evaluating all configurations equally.

```
from sklearn.experimental import enable_halving_search_cv  # noqa
from sklearn.model_selection import HalvingRandomSearchCV
from scipy.stats import loguniform

param_dist = {
  &quot;lr__C&quot;: loguniform(1e-3, 1e3)
}

halving_search = HalvingRandomSearchCV(
  estimator=model_lr,
  param_distributions=param_dist,
  # --- Successive halving controls ---
  factor=3,
  resource=&quot;n_samples&quot;,
  min_resources=2000,      # ensure enough data per fit to preserve class balance
  max_resources=len(X_train),
  # --- Cross-validation and scoring ---
  cv=skf,
  scoring=&quot;roc_auc&quot;,
  # --- Performance &amp; reproducibility ---
  n_jobs=-1,
  random_state=SEED
)

halving_search.fit(X_train, y_train)
halving_search.best_params_, halving_search.best_score_


# Output:
# /usr/local/lib/python3.12/dist-packages/sklearn/impute/_base.py:635: UserWarning:
Skipping features without any observed values: ['term' 'emp_length_years']. At least one
non-missing value is needed for imputation with strategy='median'.
#   warnings.warn(
#   ({'lr__C': np.float64(0.35836737299092375)}, np.float64(0.9554116116258321))
```

The output shows the hyperparameter configuration that survived all rounds of successive halving and achieved the highest mean cross-validated ROC AUC.

In this run, the best-performing configuration corresponds to a logistic regression model with an inverse regularization strength (*C*) of approximately 0.36, producing a mean ROC AUC of about 0.955 across the stratified cross-validation folds.

This score is directly comparable to the results from *GridSearchCV* and *RandomizedSearchCV* earlier in the chapter. The difference lies not in the metric being optimized, but in how computational effort is allocated during the search.

Successive halving introduces several parameters that do not appear in grid or randomized search. These parameters control how training resources are allocated and how aggressively poor-performing configurations are eliminated.

- **resource="n_samples"**: Specifies that model complexity is increased by giving surviving configurations access to more training data in later rounds, rather than by increasing iterations or model depth.
- **min_resources**: Sets the minimum number of training samples used in the earliest round. Here, it is set to 2000 to ensure that each fold contains enough observations from both classes, preventing unstable fits or single-class failures.
- **max_resources**: Defines the maximum amount of training data used in the final round. By setting this to the full size of the training set, the final surviving configurations are evaluated under realistic, production-level conditions.
- **factor**: Controls how aggressively the search eliminates configurations at each stage. With *factor=3*, roughly two-thirds of configurations are discarded per round, allowing the search to focus quickly on promising regions of the hyperparameter space.

Compared to *RandomizedSearchCV*, successive halving introduces an explicit resource-allocation strategy: weak configurations are unlikely to consume large computational budgets because they are filtered out early. Compared to *GridSearchCV*, it avoids the exponential cost of fully evaluating every possible combination.

The key idea is that successive halving changes *how* computation is spent, not *what* is being optimized. You still select hyperparameters based on cross-validated performance, but you do so in a way that mirrors real-world workflows: inexpensive experiments first, careful evaluation only for the most promising candidates.

Finally, it may be useful to get some advice on how you might balance the trade-off between computational cost and performance. You'll find that below along with a reminder of common hyperparameters for each model type and how to choose their ranges.

Compute Budget Guidance

- Start with RandomizedSearchCV when tuning more than three hyperparameters.
- Reduce search ranges before increasing *n_iter*.
- Use fewer folds (for example, 3) during early exploration, then 5–10 folds for final selection.
- Track training time alongside performance metrics.
- Stop searching when validation performance plateaus.
- Use *n_jobs=-1* to parallelize whenever possible.

Common hyperparameters by model type

If you don't remember all hyperparameters for each model type, here are some common ones you might use:

- **Decision Trees**: max_depth, min_samples_split, min_samples_leaf
- **Random Forest**: n_estimators, max_depth, max_features, min_samples_leaf
- **Gradient Boosting**: n_estimators, learning_rate, max_depth
- **Logistic Regression**: C (regularization), penalty
- **Support Vector Machines**: C, kernel, gamma

Choosing hyperparameter ranges

- **C**: use logarithmic scales (for example, 1e-3 to 1e3).
- **max_depth**: start small (2–10) and increase cautiously.
- **min_samples_leaf**: try small integers or small percentages of the dataset.
- **n_estimators**: tune other parameters first, then increase if needed.
- **learning_rate**: smaller values often require more estimators.

At this stage, you have a complete toolkit for systematic hyperparameter tuning. In the next section, you will use these tools to compare multiple algorithms fairly and select a final model based on performance, stability, and interpretability.

15.7Model Selection

Once hyperparameters have been tuned, the next step is *model selection*: deciding which algorithm to use moving forward. This decision should never be based on a single train/test split or a single performance number. Instead, models must be compared using a fair, consistent evaluation framework.

A fair comparison ensures that observed performance differences reflect genuine differences in model behavior rather than artifacts of data leakage, preprocessing choices, or random variation in the evaluation process.

Fair Comparison Framework

To compare models fairly, all candidates must be evaluated under the same conditions. This principle applies regardless of whether you are comparing logistic regression to a decision tree or comparing multiple ensemble methods.

- **Same data splits:** All models must use the same train/test split or the same cross-validation folds. This is why reusing a fixed *random_state* or a shared cross-validation object (such as *StratifiedKFold*) is critical.
- **Same preprocessing pipeline:** Feature scaling, imputation, and encoding must be identical across models. Pipelines make this enforceable and prevent accidental leakage.
- **Appropriate tuning for each model:** Each algorithm should be tuned using hyperparameters that matter for that model (for example, *C* for logistic regression, *max_depth* for trees).
- **Same evaluation metrics:** Models must be compared using the same scoring metrics. Mixing accuracy, ROC AUC, and F1 without a clear primary metric leads to inconsistent conclusions.

If any of these conditions change between models, the comparison is no longer valid. Small performance differences are especially sensitive to inconsistencies in evaluation setup.

Interpreting Performance Differences

A common question during model selection is whether a small difference in performance is meaningful. For example, is an accuracy of 0.85 truly better than 0.84?

Cross-validation provides more than just an average score—it also provides a measure of variability. The standard deviation across folds reflects how sensitive the model's performance is to changes in the training data.

- **Mean performance:** Represents expected performance on new data.
- **Standard deviation:** Indicates stability. High variance suggests the model's performance depends strongly on which samples it sees.

As a practical rule of thumb, a performance difference should exceed roughly *two times the standard deviation* to be considered meaningfully different in applied settings. Smaller differences are often within normal sampling noise.

Formal statistical tests such as paired t-tests or Wilcoxon signed-rank tests can be used to compare models rigorously. These methods are typically covered in statistics courses and are most relevant for research reporting rather than routine business decision-making.

Building a Comparison Table

A clear comparison table helps summarize results and support transparent decision-making. This format builds directly on the classification and ensemble comparison tables introduced in earlier chapters.

A typical comparison table includes the following columns:

- **Model:** Algorithm name (for example, Logistic Regression, Random Forest, Gradient Boosting).
- **Accuracy (mean ± std):** Cross-validated accuracy with variability.

- **F1 score:** Especially important for imbalanced classification problems.
- **ROC AUC:** Measures ranking quality across thresholds.
- **Training time:** Helps balance performance gains against computational cost.

Tables should be sorted by the primary metric that aligns with the business or analytical objective. Secondary metrics are then used as tie-breakers or to highlight trade-offs such as interpretability, stability, or speed.

The goal of model selection is not to chase the highest possible score at all costs. Instead, it is to choose a model that performs reliably, generalizes well, aligns with project constraints, and supports downstream decision-making.

Let's create a sample comparison table for the models we have been working with:

Table 15.3
Example model comparison table (cross-validated results)

Model	Accuracy (mean ± std)	F1	ROC AUC	Training time (s)	Notes
Logistic Regression	0.842 ± 0.018	0.61	0.956	1.2	Strong baseline; highly interpretable and fast. Good probability calibration.
Decision Tree	0.831 ± 0.042	0.58	0.921	0.4	Simple and interpretable, but higher variance across folds.
Random Forest	0.856 ± 0.014	0.64	0.963	9.8	Improved performance and stability; reduced interpretability.

Model	Accuracy (mean ± std)	F1	ROC AUC	Training time (s)	Notes
Gradient Boosting	0.861 ± 0.013	0.66	0.968	18.5	Best overall performance; higher training cost.

Based on the example comparison table, a reasonable default choice would be *Gradient Boosting* because it shows the strongest overall discrimination (highest ROC AUC) while also improving F1, which reflects better balance between precision and recall. However, model selection is not only about the highest score: if interpretability or training speed are priorities, Logistic Regression or a simpler tree-based model may be preferable even with slightly lower performance.

Avoid "winner-take-all" thinking: If the top two models have mean scores that are within about *one standard deviation* of each other, the difference may be mostly due to sampling noise across folds rather than a truly better algorithm. In that case, prefer the model that is simpler, faster, easier to explain, or more stable—unless there is a clear business reason to accept extra complexity.

Table 15.4
Tuning the selected model (example hyperparameter comparison)

Configuration	Hyperparameters	Accuracy (mean ± std)	F1	ROC AUC	Training time (s)
GB-1 (baseline)	n_estimators=200; learning_rate=0.10; max_depth=3	0.861 ± 0.013	0.66	0.968	18.5

Configuration	Hyperparameters	Accuracy (mean ± std)	F1	ROC AUC	Training time (s)
GB-2 (more estimators, smaller steps)	n_estimators=500; learning_rate=0.05; max_depth=3	0.864 ± 0.012	0.67	0.970	44.1
GB-3 (shallower trees)	n_estimators=500; learning_rate=0.05; max_depth=2	0.862 ± 0.012	0.66	0.969	34.7
GB-4 (deeper trees)	n_estimators=200; learning_rate=0.10; max_depth=4	0.859 ± 0.015	0.65	0.965	26.9
GB-5 (more estimators, slightly larger trees)	n_estimators=500; learning_rate=0.05; max_depth=4	0.863 ± 0.014	0.66	0.967	61.3

The second table shows the next step after choosing a strong algorithm: *tuning*. Instead of comparing different model families, you hold the algorithm fixed and compare several plausible hyperparameter configurations using the same cross-validation strategy and scoring metrics. This is how teams turn a “good model choice” into a “production-quality configuration.”

Interpret the results by focusing on three ideas at once: *(1) validation performance*, *(2) stability* (the standard deviation across folds), and *(3) computational cost*. For example, a configuration that improves ROC AUC by only 0.001 but doubles training time may not be worth it—especially if the improvement is smaller than the typical fold-to-fold variation.

A practical decision rule is to favor the simplest configuration that performs near the top. In this example, GB-2 has the best mean ROC AUC, but it is substantially slower. If GB-1 or GB-3 performs within one standard deviation of GB-2, many teams would choose the faster option unless the business value of the small performance gain is clearly justified.

Finally, notice that tuning does not always produce dramatic gains. Sometimes the “best” configuration is only slightly better than the baseline, which is a useful finding: it suggests the algorithm is relatively robust, and future improvements may come from better features, different model families, or changes to the training data rather than continued micro-tuning of hyperparameters.

So where do we begin? Do we just test every possible algorithm with as much tuning as we have time and compute resources for? Actually, there is some decision guidance we can refer to in the next section that can narrow the scope of the modeling phase.

15.8Decision Guide

Now that you've been introduced to critical concepts like cross-validation and hyperparameter tuning, this section provides some practical guidance to help you navigate algorithm selection in a disciplined way. The goal is not to memorize rules, but to develop sound judgment about which models are reasonable starting points and which are unlikely to be a good fit.

Decision Framework

- **Start simple:** Begin with a logistic regression or a single decision tree as a baseline. Simple models are fast to train, easy to diagnose, and often

surprisingly competitive. This will also give you feature coefficients to evaluate for key influencers analysis.

- **Interpretability required?** If explanations are required for regulation, auditing, or stakeholder trust, favor linear models, decision trees, or rule-based approaches.
- **Large dataset (> 100k rows)?** Gradient boosting and neural networks become more viable as data volume grows. These models often benefit from large sample sizes but are more computationally expensive.
- **Small dataset (< 1k rows)?** Favor simpler models with strong regularization. Complex models are more likely to overfit when data is scarce.
- **High-dimensional data (features > samples)?** Regularization is essential. Linear models with L1 or L2 penalties are often strong choices in this setting.
- **Mixed feature types?** Tree-based models naturally handle mixtures of numeric and categorical variables and complex interactions with minimal feature engineering.
- **Need probabilities?** If decisions depend on predicted probabilities (risk scores, ranking, thresholds), probability quality matters as much as classification accuracy.

Probability Quality and Calibration

In earlier chapters, you used metrics such as log loss to evaluate classification models. Log loss directly measures *probability quality*, not just whether predictions are correct.

If a model's outputs will be used as probabilities—for example, to rank customers by risk or to trigger actions at specific thresholds—optimizing

accuracy alone can be misleading. A model can have high accuracy while producing poorly calibrated probabilities.

- Optimize or tune using *neg_log_loss* when probabilities matter.
- Compare models using probability-based metrics such as log loss or average precision in addition to accuracy or ROC AUC.
- Consider probability calibration techniques when necessary. Calibration curves and methods such as Platt scaling can be useful (research on your own).

A key insight is that strong ranking performance does not guarantee good probability estimates. Log loss helps reveal this difference.

Model characteristics summary

Table 15.5
Common classification models and their characteristics

Model	Interpretable?	Handles nonlinearity?	Handles interactions?	Fast training?	Fast inference?
Logistic Regression	Yes	No	Manual	Yes	Yes
Decision Tree	Yes	Yes	Yes	Yes	Yes
k-NN	Limited	Yes	Yes	Fast	Slow
SVM	Limited	Yes (kernels)	Yes (kernels)	Slow	Fast
Random Forest	Limited	Yes	Yes	Medium	Medium
Gradient Boosting	Limited	Yes	Yes	Slow	Fast
XGBoost	Limited	Yes	Yes	Medium	Fast

Performance vs. interpretability trade-offs

In many real-world projects, the highest-performing model is not automatically the best choice. Regulated environments, stakeholder trust, and operational constraints often favor simpler or more interpretable models.

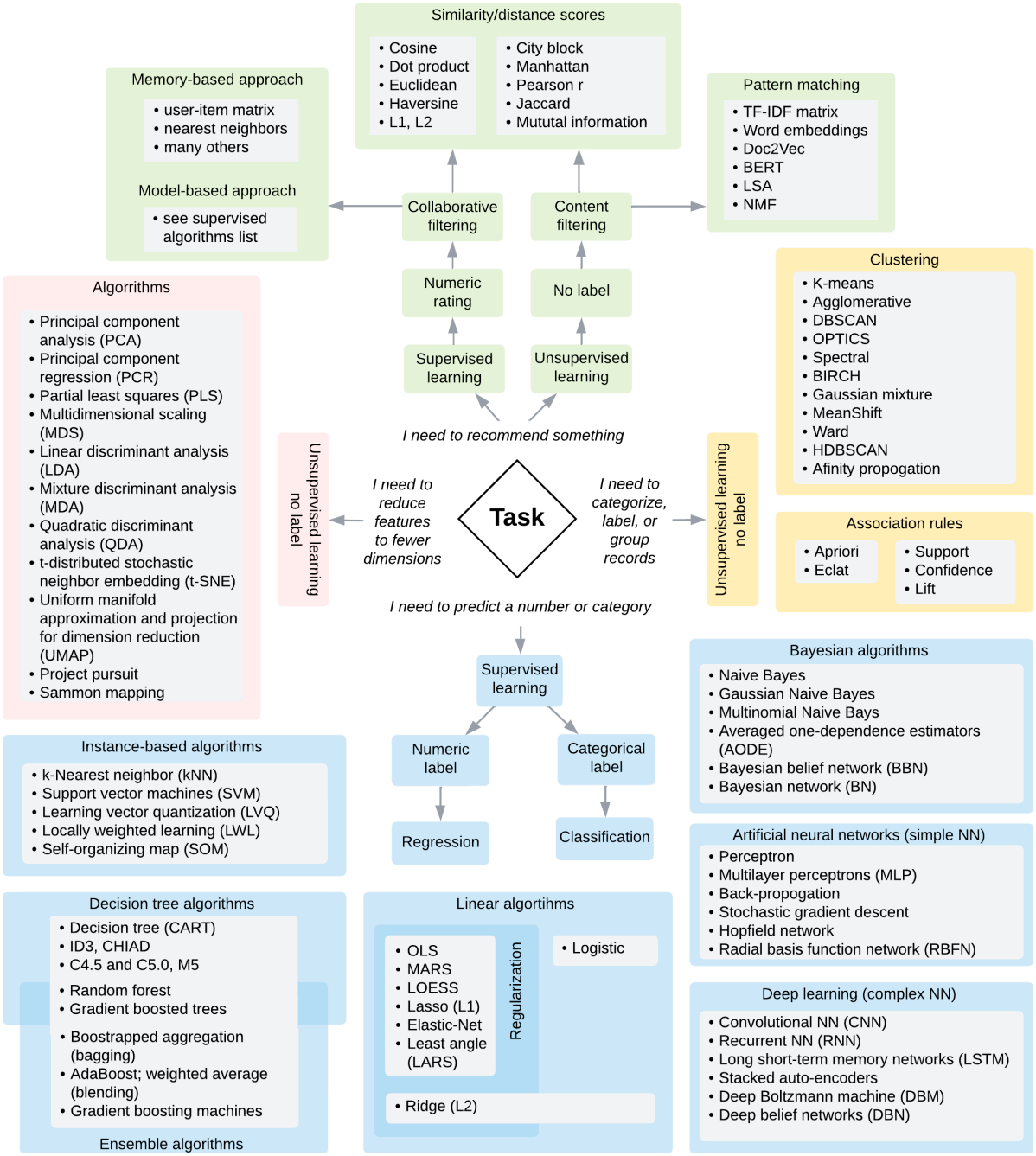
When performance differences are small relative to cross-validation variability, choosing a simpler model can be the more responsible decision. Complexity should be justified by meaningful, stable performance gains.

At this stage, your focus is on selecting a strong model family and tuning it responsibly. Feature selection and feature refinement are still part of the modeling lifecycle, but they are addressed explicitly in the next chapter to keep the learning process structured and manageable.

Decision Framework

The figure below provides a “big picture” map of common machine learning problem types and the algorithm families that are often used for them. You are *not* expected to know most of the algorithms shown yet. Instead, use this diagram as a decision guide: it helps you start with the right *problem framing* (Do I have labels? Am I predicting a number or category? Do I need probabilities?) before you worry about specific model names.

In this chapter, you have worked primarily in the *supervised learning* portion of the map (classification with logistic regression and decision trees, plus ensembles). The value of the figure is that it shows where those models “live” relative to other approaches you will encounter later, such as clustering, dimensionality reduction, and recommender systems.



Big-picture decision map: start by identifying your task type, then choose an algorithm family (and only then worry about specific models and hyperparameters).

How to read this diagram

Use the diagram from the center outward. Start by stating your task in plain language (for example, “I need to predict default risk” or “I need to group

similar customers”). Then answer the key branching question: *Do I have labels?* If yes, you are in supervised learning; if no, you are in unsupervised learning.

If you have labels, decide whether your target is *numeric* (regression) or *categorical* (classification). From there, use practical constraints to narrow the choice: interpretability requirements, dataset size, feature types, and whether you need well-calibrated probabilities for decision thresholds.

Treat the algorithm lists as *families*, not a checklist. In this chapter, you focused on a few core families (logistic regression, decision trees, and ensembles) and learned how to evaluate and tune them correctly. In a later chapter, you will revisit the same workflow after you learn feature selection, which can change performance as much as changing the algorithm.

15.9Evaluation Workflow

A complete evaluation workflow with a three-class target

In this section, you will apply the complete evaluation workflow from this chapter end-to-end using the Lending Club dataset. To make the workflow feel meaningfully different from earlier binary examples, you will frame the task as a *three-class classification problem* that matches common credit-risk triage: *good* outcomes, *late* outcomes, and *bad* outcomes.

The goal is not to memorize a single “best” model. Instead, the goal is to practice a professional sequence of steps: freeze a test set, build a baseline pipeline, evaluate with cross-validation, diagnose with curves, tune systematically, compare fairly, validate on the untouched test set, and report results in a way that supports decisions.

Because the target is now multiclass, you will also see why metric choice matters. Some metrics (like accuracy) can be dominated by the majority class. Others (like macro-averaged F1) treat each class equally, which is often more appropriate when the “late” class is rare but business-relevant.

Step 1: Import and clean the data

We start by importing the dataset and applying the same cleaning steps you used earlier. These steps are intentionally repeated here so you can see how professional workflows remain consistent across tasks, even when the target definition changes.

```
import numpy as np
import pandas as pd

SEED = 27
DATA_PATH = &quot;/content/drive/MyDrive/Colab Notebooks/data/lc_small.csv&quot;
df = pd.read_csv(DATA_PATH)

# Drop columns used in prior chapter
drop_cols = [&quot;loan_status_numeric&quot;, &quot;emp_title&quot;,
&quot;title&quot;]
drop_cols = [c for c in drop_cols if c in df.columns]
df = df.drop(columns=drop_cols)

# Convert issue date to &quot;age&quot; feature (days since most recent issue date)
if &quot;issue_d&quot; in df.columns:
  df[&quot;issue_d&quot;] = pd.to_datetime(df[&quot;issue_d&quot;],
errors=&quot;coerce&quot;)
  max_issue_date = df[&quot;issue_d&quot;].max()
  df[&quot;issue_age_days&quot;] = (max_issue_date - df[&quot;issue_d&quot;]).dt.days
  df = df.drop(columns=[&quot;issue_d&quot;])

# Parse term to numeric months
if &quot;term&quot; in df.columns:
  df[&quot;term&quot;] =
df[&quot;term&quot;].astype(str).str.strip().str.extract(r&quot;(\d+)&quot;).astype(float)

# Parse employment length to numeric years
if &quot;emp_length&quot; in df.columns:
  emp = df[&quot;emp_length&quot;].astype(str).str.strip()
  emp = emp.replace({&quot;nan&quot;: np.nan, &quot;None&quot;: np.nan})
  emp = emp.replace({&quot;10+ years&quot;: &quot;10&quot;, &quot;&lt; 1 year&quot;:
&quot;0&quot;})
  emp = emp.str.extract(r&quot;(\d+)&quot;)[0]
  df[&quot;emp_length_years&quot;] = pd.to_numeric(emp, errors=&quot;coerce&quot;)
  df = df.drop(columns=[&quot;emp_length&quot;])
```

```
# Missingness indicators for selected delinquency/record columns
for col in [&quot;mths_since_last_delinq&quot;, &quot;mths_since_last_record&quot;]:
  if col in df.columns:
    ind_col = col + &quot;_missing&quot;
    df[ind_col] = df[col].isna().astype(int)
    max_val = df[col].max(skipna=True)
    fill_val = (max_val + 1) if pd.notna(max_val) else 0
    df[col] = df[col].fillna(fill_val)

df.shape


# Output:
# (10476, 34)
```

After cleaning, your dataset is in a consistent “model-ready” shape: unnecessary text columns are removed, date information is converted into a numeric age feature, common messy fields are parsed into numeric form, and select missingness patterns are captured explicitly. These kinds of repeatable cleaning steps are exactly what makes an evaluation workflow reliable.

Step 2: Define a three-class target

Next, you will convert the original loan status into three classes. The mapping below is designed to match a realistic triage use case:

- **bad**: loans that ended in a clear negative outcome such as *Charged Off* or *Default*.
- **good**: loans that are performing or completed successfully such as *Current* or *Fully Paid*.
- **late**: all other statuses, which often represent delinquency stages, hardship, or uncertain outcomes that are operationally important even if they are not final “defaults.”

This framing is useful because it separates the world into “healthy,” “clearly bad,” and “needs attention.” In practice, the late category often triggers early interventions, collections workflows, or different underwriting rules.

```
      # Require loan_status for the multiclass target
      if &quot;loan_status&quot; not in df.columns:
        raise ValueError(&quot;Expected a 'loan_status' column for the multiclass
target.&quot;)

      # Define status groups
      bad_statuses = {&quot;Charged Off&quot;, &quot;Default&quot;}
      good_statuses = {&quot;Current&quot;, &quot;Fully Paid&quot;}

      # Drop rows with missing loan_status (a small number of rows can otherwise poison
evaluation)
      df = df.dropna(subset=[&quot;loan_status&quot;]).copy()

      def map_status_3class(s):
        s = str(s).strip()

        if s in bad_statuses:
          return &quot;bad&quot;

        if s in good_statuses:
          return &quot;good&quot;

        return &quot;late&quot;

      df[&quot;loan_status_3class&quot;] =
df[&quot;loan_status&quot;].apply(map_status_3class)
      df[&quot;loan_status_3class&quot;].value_counts(dropna=False)


      # Output:
      # count
      # loan_status_3class
      # good    9334
      # bad     898
      # late    244
      # dtype: int64
```

The output confirms that the multiclass target is highly imbalanced: most loans fall into the *good* category (9,334 observations), while the *bad* class is much smaller (898 observations) and the *late* class is very rare (244 observations). This imbalance has important consequences for the rest of the workflow. You must use *stratified cross-validation* so that all three classes appear in every fold, and you should avoid relying on overall accuracy alone. A model can achieve high accuracy simply by predicting the dominant *good* class while performing poorly on *bad* or *late* loans. For this reason, later steps in this section emphasize *balanced accuracy*, which gives equal weight to each class and provides a more honest assessment of whether the model is

learning useful distinctions across all outcome categories rather than optimizing performance on the majority class.

Step 3: Freeze a final test set

To prevent accidental overfitting to evaluation results, you should freeze a final test set early. You will use the training set for cross-validation, diagnostics, and tuning. The test set remains untouched until the end of the workflow.

```
from sklearn.model_selection import train_test_split

y = df[&quot;loan_status_3class&quot;].copy()
X = df.drop(columns=[&quot;loan_status&quot;, &quot;loan_status_3class&quot;]).copy()

X_train, X_test, y_train, y_test = train_test_split(
  X,
  y,
  test_size=0.2,
  random_state=SEED,
  stratify=y
)

X_train.shape, X_test.shape, y_train.value_counts()


# Output:
# ((8380, 33),
#  (2096, 33),
#  loan_status_3class
#  good    7467
#  bad      718
#  late     195
#  Name: count, dtype: int64
```

The output shows that stratification successfully preserved the class proportions in both splits. In the training set (8,380 rows), the class counts closely mirror the full dataset: most observations are *good* loans (7,467), while *bad* (718) and especially *late* loans (195) remain much smaller groups. This balance is critical for reliable model development. Stratification ensures that every class appears in both the training and test sets, which allows cross-

validation to function correctly and makes final evaluation meaningful. Without stratification, a random split could easily produce a test set with very few—or even zero—examples of the rare *late* class, leading to misleading performance metrics and unstable conclusions.

Step 4: Build a reusable preprocessing pipeline

A key theme of this chapter is that preprocessing must be performed *inside* cross-validation. The safest way to do that is to place all preprocessing steps in a scikit-learn pipeline. That way, each cross-validation fold fits preprocessing using only the training portion of that fold, which prevents data leakage.

This pipeline uses a common professional pattern: numeric features are imputed and scaled, categorical features are imputed and one-hot encoded, and the two branches are combined with a ColumnTransformer. This structure works well for most tabular business datasets.

```
from sklearn.compose import ColumnTransformer, make_column_selector as selector
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer

numeric_features = selector(dtype_include=&quot;number&quot;)
categorical_features = selector(dtype_exclude=&quot;number&quot;)

numeric_transformer = Pipeline(steps=[
  (&quot;imputer&quot;, SimpleImputer(strategy=&quot;median&quot;)),
  (&quot;scaler&quot;, StandardScaler())
])

categorical_transformer = Pipeline(steps=[
  (&quot;imputer&quot;, SimpleImputer(strategy=&quot;most_frequent&quot;)),
  (&quot;onehot&quot;, OneHotEncoder(handle_unknown=&quot;ignore&quot;))
])

preprocess = ColumnTransformer(
  transformers=[
    (&quot;num&quot;, numeric_transformer, numeric_features),
    (&quot;cat&quot;, categorical_transformer, categorical_features)
  ],
```

```
  remainder=&quot;drop&quot;
)

preprocess
```

In a workflow like this, *preprocess* becomes a reusable component: you can swap different models at the end of the pipeline while keeping the data preparation identical. That makes comparisons fair because differences in performance reflect the model choice rather than inconsistent preprocessing.

Step 5: Baseline model and cross-validated evaluation

With preprocessing in place, you can build a baseline classifier. Logistic regression is a common baseline because it is fast, widely used in industry, and provides reasonably strong performance on many tabular problems. For multiclass problems, logistic regression can be trained in a multinomial setting so that all classes are modeled together.

You will evaluate the baseline using stratified K-fold cross-validation. Cross-validation produces a distribution of scores rather than a single number, which helps you understand both the typical performance (mean) and the variability (standard deviation).

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_validate

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)

model_lr_mc = Pipeline(steps=[
  (&quot;preprocess&quot;, preprocess),
  (&quot;lr&quot;, LogisticRegression(
    max_iter=3000,
    solver=&quot;lbfgs&quot;
  ))
])

scoring = {
  &quot;accuracy&quot;: &quot;accuracy&quot;,
  &quot;balanced_accuracy&quot;: &quot;balanced_accuracy&quot;,
  &quot;f1_macro&quot;: &quot;f1_macro&quot;,
```

```
    &quot;f1_weighted&quot;: &quot;f1_weighted&quot;
  }

  cv_results = cross_validate(
    model_lr_mc,
    X_train,
    y_train,
    cv=skf,
    scoring=scoring,
    n_jobs=-1,
    return_train_score=False
  )

  {key: (np.mean(val), np.std(val)) for key, val in cv_results.items() if
key.startswith(&quot;test_&quot;)}


  # Output:
  # {'test_accuracy': (np.float64(0.9335322195704057),
  # np.float64(0.004986296519278308)),
  # 'test_balanced_accuracy': (np.float64(0.5568052667356447),
  # np.float64(0.017711473953789338)),
  # 'test_f1_macro': (np.float64(0.5880255268882838),
  # np.float64(0.027500220813195482)),
  # 'test_f1_weighted': (np.float64(0.9225260199981079),
  # np.float64(0.005850658305429231))}
```

The results illustrate why relying on a single metric can be misleading in multiclass problems with imbalanced classes. Overall accuracy is very high (about 0.93), which largely reflects the model's strong performance on the dominant *good* class. However, balanced accuracy drops sharply to around 0.56, revealing that performance across the three classes is uneven. Macro-averaged F1 (approximately 0.59) tells a similar story by weighting each class equally, regardless of how common it is. Together, these metrics show that while the baseline logistic regression appears strong at first glance, it struggles to treat *bad* and *late* loans as carefully as *good* ones. This gap motivates the use of balanced accuracy later in the workflow, since it better reflects performance in settings where each class represents a distinct and operationally meaningful outcome.

Step 6: Diagnose with a learning curve

Next, use a learning curve to diagnose whether the baseline model is limited by data size (high variance) or by model capacity (high bias). Learning curves

compare training performance and cross-validated validation performance as you increase the training set size.

For multiclass classification, it is common to use a single scoring metric for curves to keep interpretation simple. Here we use *balanced accuracy* because it accounts for class imbalance by averaging recall across classes.

```
from sklearn.model_selection import learning_curve
import matplotlib.pyplot as plt

train_sizes, train_scores, val_scores = learning_curve(
  model_lr_mc,
  X_train,
  y_train,
  cv=skf,
  scoring=&quot;balanced_accuracy&quot;,
  train_sizes=np.linspace(0.1, 1.0, 10),
  n_jobs=-1
)

train_mean = train_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
val_mean = val_scores.mean(axis=1)
val_std = val_scores.std(axis=1)
plt.figure(figsize=(8, 5))
plt.plot(train_sizes, train_mean, marker=&quot;o&quot;, label=&quot;Training
(mean)&quot;)
plt.plot(train_sizes, val_mean, marker=&quot;o&quot;, label=&quot;Validation (CV
mean)&quot;)
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std,
alpha=0.2)
plt.fill_between(train_sizes, val_mean - val_std, val_mean + val_std, alpha=0.2)
plt.xlabel(&quot;Training set size&quot;)
plt.ylabel(&quot;Balanced accuracy&quot;)
plt.title(&quot;Learning Curve: Multiclass Logistic Regression&quot;)
plt.grid(True)
plt.legend()
plt.show()
train_sizes, train_mean, val_mean


# Output:
# (array([ 670, 1340, 2011, 2681, 3352, 4022, 4692, 5363, 6033, 6704]),
# array([0.72675797, 0.66434462, 0.61953078, 0.61764323, 0.60816766,

# 0.60594154, 0.60080069, 0.59486597, 0.5848202 , 0.58734778]),
# array([0.49795647, 0.53937189, 0.5364115 , 0.53499335, 0.53776316,

# 0.53728184, 0.54113575, 0.54507723, 0.54778345, 0.55680527]))
```

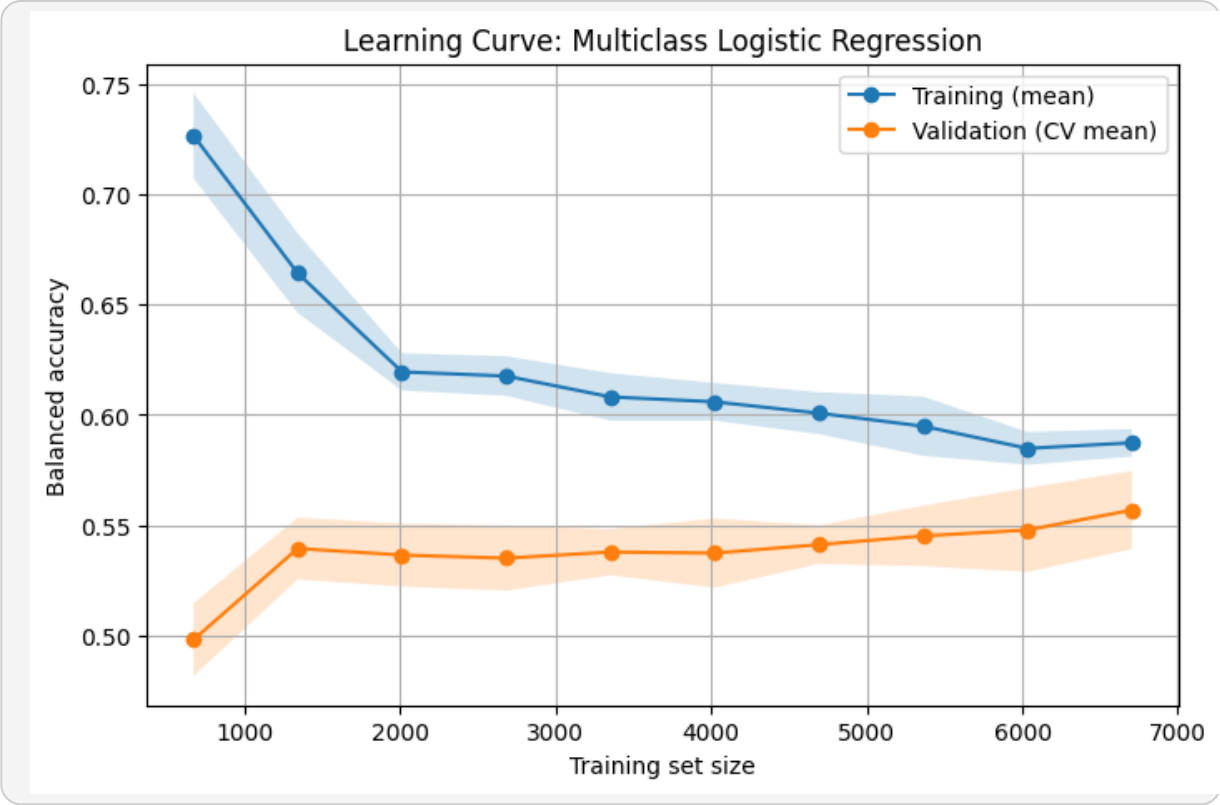


Figure 15.1: Learning Curve

This learning curve shows a classic high-bias (underfitting) pattern. The training balanced accuracy starts relatively high (around 0.73 with small samples) but steadily declines as more data is added, settling near 0.59. At the same time, the validation balanced accuracy improves slowly from about 0.50 to roughly 0.56, but the two curves remain separated and converge at a fairly modest level. This indicates that adding more data helps somewhat, but not enough to close the gap or raise performance substantially. The takeaway is that the baseline multiclass logistic regression is too restrictive for this problem: regularization and linear decision boundaries limit its ability to capture the structure needed to distinguish *good*, *bad*, and *late* loans. Rather than focusing on overfitting controls, the next step should be to increase model flexibility through hyperparameter tuning and, potentially, by comparing against more expressive algorithms.

Step 7: Diagnose hyperparameter sensitivity with a validation curve

Validation curves focus on one hyperparameter at a time. Here you will examine how logistic regression responds to different values of *C*, the inverse regularization strength. Smaller *C* means stronger regularization (simpler model); larger *C* means weaker regularization (more flexible model).

```
from sklearn.model_selection import validation_curve

param_range = np.logspace(-3, 3, 7)

train_scores, val_scores = validation_curve(
  model_lr_mc,
  X_train,
  y_train,
  param_name=&quot;lr__C&quot;,
  param_range=param_range,
  cv=skf,
  scoring=&quot;balanced_accuracy&quot;,
  n_jobs=-1
)

train_mean = train_scores.mean(axis=1)
val_mean = val_scores.mean(axis=1)
plt.figure(figsize=(8, 5))
plt.plot(param_range, train_mean, marker=&quot;o&quot;, label=&quot;Training
(mean)&quot;)
plt.plot(param_range, val_mean, marker=&quot;o&quot;, label=&quot;Validation (CV
mean)&quot;)
plt.xscale(&quot;log&quot;)
plt.xlabel(&quot;Inverse regularization strength (C)&quot;)
plt.ylabel(&quot;Balanced accuracy&quot;)
plt.title(&quot;Validation Curve: Multiclass Logistic Regression&quot;)
plt.grid(True)
plt.legend()
plt.show()
param_range, train_mean, val_mean


# Output:
# (array([1.e-03, 1.e-02, 1.e-01, 1.e+00, 1.e+01, 1.e+02, 1.e+03]),
# array([0.34279942, 0.46306281, 0.53584018, 0.58734778, 0.6302593 ,

# 0.64939947, 0.65619826]),
# array([0.34062947, 0.46108182, 0.51816789, 0.55680527, 0.56042837,

# 0.55727073, 0.55339495]))
```

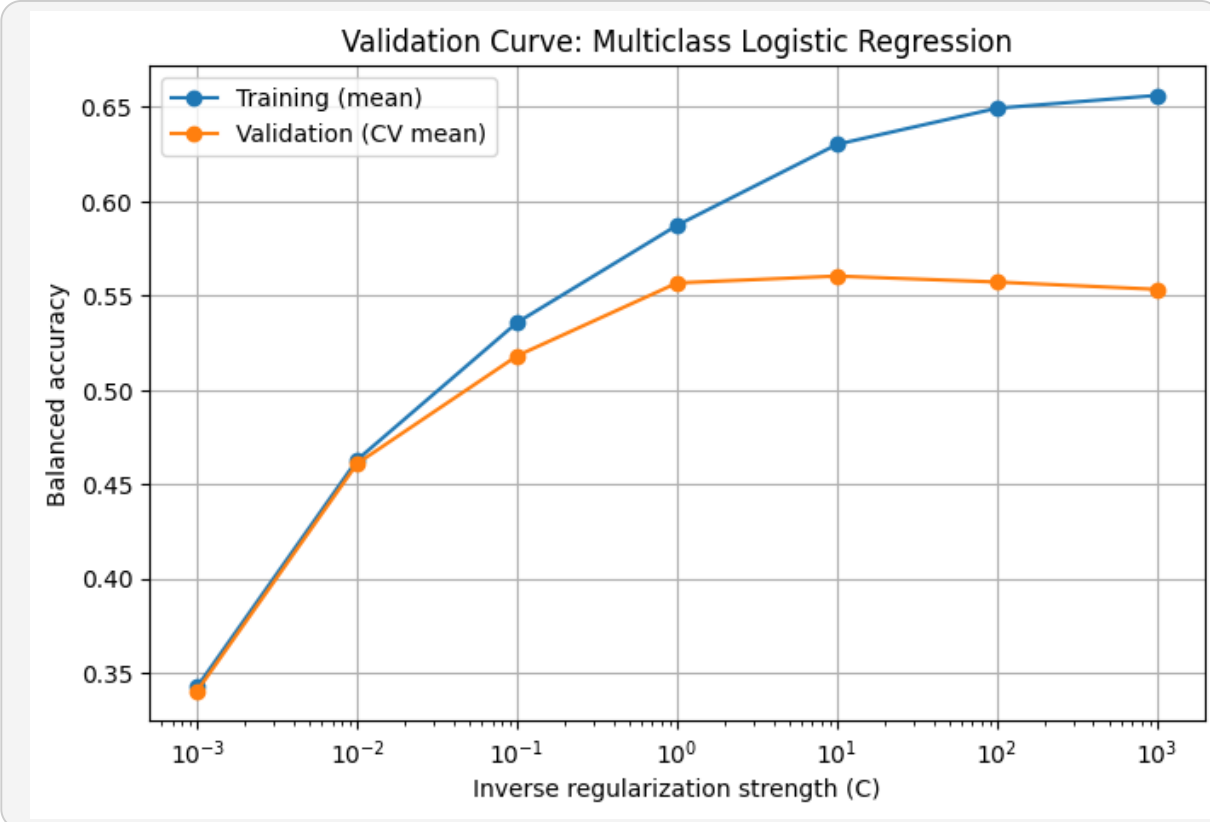


Figure 15.2: Validation Curve

This validation curve shows that balanced accuracy improves rapidly as C increases from very small values, indicating severe underfitting when regularization is too strong. Validation performance peaks around $C \approx 1$ *to* 10, then flattens and begins to decline slightly at larger values, even as training performance continues to increase. This widening gap between training and validation scores at high C signals the onset of overfitting as regularization weakens. The curve therefore suggests a practical search range centered on moderate values of C, rather than extreme ends of the scale. This is exactly the role of validation curves: they narrow the hyperparameter space so that grid search or randomized search can focus computation where generalization performance is most likely to improve.

Step 8: Tune hyperparameters systematically

Once you have a baseline and diagnostics, the next step is systematic tuning. Instead of adjusting one parameter at a time, you search combinations in a controlled way using cross-validation. This reduces the risk of overfitting to a single validation split and helps you discover settings that work well together.

GridSearchCV

GridSearchCV evaluates every combination in a predefined grid. It is best when you have a small number of hyperparameters and narrow, informed ranges. The example below tunes *C* (regularization strength) and *l1_ratio* (regularization type: 0.0 for pure L2, 1.0 for pure L1).

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
import warnings
from sklearn.exceptions import ConvergenceWarning

# Optional: suppress convergence warnings during grid search only
warnings.filterwarnings(&quot;ignore&quot;, category=ConvergenceWarning)

model_lr_mc_grid = Pipeline(steps=[
  (&quot;preprocess&quot;, preprocess),
  (&quot;lr&quot;, LogisticRegression(
    penalty=&quot;elasticnet&quot;,   # needed for sklearn &lt;1.10; remove once
sklearn ≥1.10
    solver=&quot;saga&quot;,          # supports all l1_ratio values
    max_iter=5000,          # higher ceiling for hard cases
    tol=1e-3,               # relax convergence slightly for grid search
    n_jobs=1                # avoid thread oversubscription
  ))
])

param_grid = {
  &quot;lr__C&quot;: [0.01, 0.1, 1.0, 10.0],
  &quot;lr__l1_ratio&quot;: [0.0, 1.0]   # 0.0 = L2, 1.0 = L1
}

grid_search = GridSearchCV(
  estimator=model_lr_mc_grid,
  param_grid=param_grid,
  cv=skf,
  scoring=&quot;balanced_accuracy&quot;,
  n_jobs=-1,
  return_train_score=True
)
```

```
grid_search.fit(X_train, y_train)
grid_search.best_params_, grid_search.best_score_


# Output:
# ({'lr__C': 10.0, 'lr__l1_ratio': 1.0}, np.float64(0.5432173719340134))
```

Grid search identifies *C = 10.0* with *l1_ratio = 1.0* (pure L1 regularization) as the best-performing configuration under balanced accuracy, achieving a mean cross-validated score of approximately *0.54*. This result is consistent with the earlier validation curve, which showed that moderate-to-large values of *C* improved validation performance before overfitting began. The choice of L1 regularization suggests that some feature sparsity or implicit feature selection is beneficial for this multiclass problem. Importantly, this score is based entirely on cross-validation within the training data, making it appropriate for model selection while keeping the test set untouched.

Notice that the best cross-validated balanced accuracy is only modestly higher than the baseline model. This reinforces a key lesson: hyperparameter tuning often yields incremental improvements rather than dramatic gains, especially for linear models. At this point, you should compare this tuned model against other algorithms using the same evaluation framework, or consider whether the remaining performance gap reflects model bias that cannot be solved by tuning alone.

If you encounter warnings about sparse or missing features during cross-validation, they indicate that some columns are entirely absent in certain training folds. This is not an error, but a signal about data quality and feature stability. In real-world modeling workflows, such warnings prompt decisions about feature redesign, alternative imputation strategies, or dropping unreliable features. For this workflow, the warnings do not invalidate the results, but they highlight issues that would be addressed before deployment.

RandomizedSearchCV

RandomizedSearchCV samples a fixed number of random hyperparameter combinations. This is often more practical than grid search when you have many hyperparameters or wide ranges, because you can control the computational budget directly.

```
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from scipy.stats import loguniform

# Creating a simpler CV model to speed things up; normally, we'd like to keep this to
n_splits = 5 if possible
skf_fast = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)

param_dist = {
  &quot;lr__C&quot;: loguniform(1e-2, 1e2), # If you have enough time, use a wider
range: loguniform(1e-3, 1e3)
  &quot;lr__l1_ratio&quot;: [0.0]  # Pure L2; add 1.0 to also try L1 if time permits
}

random_search = RandomizedSearchCV(
  estimator=model_lr_mc_grid,
  param_distributions=param_dist,
  n_iter=12,  # If you have enough time, consider up to 30
  cv=skf_fast,
  scoring=&quot;balanced_accuracy&quot;,
  n_jobs=4,   # Sometimes this is faster than -1 in notebooks
  random_state=SEED
)

random_search.fit(X_train, y_train)
random_search.best_params_, random_search.best_score_


# Output:
# ({'lr__C': np.float64(4.493381330400462), 'lr__l1_ratio': 0.0},
# np.float64(0.5454009698485182))
```

Randomized search identifies a configuration with $C \approx 4.49$ as the best-performing setting under balanced accuracy, achieving a mean cross-validated score of about *0.55*. This result is comparable to—and slightly higher than—the grid search result, despite evaluating far fewer configurations. The improvement comes from sampling the hyperparameter space efficiently rather than exhaustively, allowing the search to focus on promising regions without paying the full computational cost of grid search.

Several deliberate simplifications were made to keep runtime reasonable: fewer cross-validation folds, a reduced number of iterations, and a narrower hyperparameter range. These choices reflect how randomized search is often used in practice—as a fast, exploratory tuning step. Once a promising range for *C* is identified, you could follow up with a more focused grid search or a higher-budget randomized search if additional performance gains are worth the extra computation.

This example highlights a key lesson in model tuning: exhaustive search is not always necessary. When training time matters, randomized search frequently finds solutions that are “good enough” or even competitive with grid search, making it a practical default for many real-world modeling workflows.

Step 9: Compare multiple algorithms fairly

Tuning is only part of model selection. In professional workflows, you typically compare multiple algorithm families using a consistent evaluation framework: the same training set, the same cross-validation strategy, the same preprocessing, and the same scoring metrics. This ensures that performance differences reflect the algorithm, not the experimental setup.

Below is a small comparison example that evaluates several candidate models using the same preprocessing and the same stratified cross-validation. This is an example of what a “comparison table” looks like in practice.

```
import time
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import make_scorer, f1_score

# Reusable scoring (multiclass)
scoring_compare = {
  &quot;balanced_accuracy&quot;: &quot;balanced_accuracy&quot;,
```

```
    &quot;f1_macro&quot;: &quot;f1_macro&quot;
  }

  candidates = [
    (&quot;LogReg (baseline)&quot;, model_lr_mc),
    (&quot;Decision Tree&quot;, Pipeline(steps=[
      (&quot;preprocess&quot;, preprocess),
      (&quot;dt&quot;, DecisionTreeClassifier(random_state=SEED))
    ])),
    (&quot;Random Forest&quot;, Pipeline(steps=[
      (&quot;preprocess&quot;, preprocess),
      (&quot;rf&quot;, RandomForestClassifier(
        n_estimators=300,
        random_state=SEED,
        n_jobs=-1
      ))
    ]))
  ]

  rows = []

  for name, estimator in candidates:
    t0 = time.time()

    res = cross_validate(
      estimator,
      X_train,
      y_train,
      cv=skf,
      scoring=scoring_compare,
      n_jobs=-1,
      return_train_score=False
    )

    elapsed = time.time() - t0

    rows.append({
      &quot;Model&quot;: name,
      &quot;Balanced Acc (mean)&quot;: res[&quot;test_balanced_accuracy&quot;].mean(),
      &quot;Balanced Acc (std)&quot;: res[&quot;test_balanced_accuracy&quot;].std(),
      &quot;F1 Macro (mean)&quot;: res[&quot;test_f1_macro&quot;].mean(),
      &quot;F1 Macro (std)&quot;: res[&quot;test_f1_macro&quot;].std(),
      &quot;CV Time (sec)&quot;: elapsed
    })

  df_compare = pd.DataFrame(rows).sort_values(by=&quot;Balanced Acc (mean)&quot;,
ascending=False)
  df_compare


  # Output:
  #         Model   Balanced Acc (mean)     Balanced Acc (std)      F1 Macro (mean) F1
Macro (std)     CV Time (sec)
  # 1       Decision Tree   0.569522        0.013175        0.574017        0.014509
3.840081
  # 0       LogReg (baseline)       0.556805        0.017711        0.588026
0.027500        2.767440
  # 2       Random Forest   0.361991        0.006134        0.367806        0.010574
55.464561
```

This comparison table illustrates why systematic evaluation matters. The decision tree achieves the highest mean balanced accuracy (≈ *0.57*), indicating slightly better average performance across the three classes than the logistic regression baseline (≈ *0.56*). However, the logistic regression model achieves a higher macro-averaged F1 score, suggesting more even precision–recall tradeoffs across classes. The random forest performs substantially worse on both metrics in this setup, despite being far more computationally expensive.

Notice the role of variability and cost in the decision. The balanced accuracy difference between the decision tree and logistic regression is smaller than one standard deviation of the cross-validation scores, which means the apparent advantage may not be stable across different samples. At the same time, the decision tree trains quickly and remains interpretable, while the random forest requires an order of magnitude more computation without delivering better performance. In situations like this, practitioners often favor the simplest model whose performance is statistically comparable—especially when interpretability, robustness, or deployment cost matters.

Step 10: Final validation on the untouched test set

After you select a final model family and finalize a reasonable hyperparameter setting, you validate once on the untouched test set. This is the moment where the workflow answers a real question: “How well will this model perform on new, unseen data?”

A clean way to do this is to take the best estimator from your search object, fit it on the full training set, and then evaluate on the test set. Because the test set was never used during selection or tuning, this evaluation is the most honest estimate of real-world generalization within the scope of your dataset.

```
from sklearn.metrics import classification_report, confusion_matrix

final_model = random_search.best_estimator_
final_model.fit(X_train, y_train)
y_pred = final_model.predict(X_test)

print(classification_report(y_test, y_pred, digits=4))
confusion_matrix(y_test, y_pred)


# Output:
#  precision    recall  f1-score   support
#
#           bad     0.8148    0.6111    0.6984       180
#          good     0.9442    0.9882    0.9657      1867
#          late     0.5714    0.0816    0.1429        49
#
#      accuracy                         0.9346      2096
#     macro avg     0.7768    0.5603    0.6023      2096
#  weighted avg     0.9244    0.9346    0.9235      2096
#
#  array([[ 110,   69,    1],
#         [  20, 1845,    2],
#         [   5,   40,    4]])
```

The final test-set results reveal an important asymmetry in model performance across classes. The model performs extremely well on the *good* class, with near-perfect recall, meaning that almost all truly good loans are correctly identified. Performance on the *bad* class is more moderate, with recall around 0.61, indicating that a meaningful portion of bad loans are still being missed. The weakest performance appears in the *late* class: recall is very low, and most late accounts are misclassified as good. The confusion matrix makes this visible by showing that late loans are overwhelmingly absorbed into the good category. This pattern is common in imbalanced multiclass problems and highlights why balanced accuracy and macro-averaged metrics were emphasized earlier—high overall accuracy alone would hide the model's limited ability to detect the rare but operationally important late class.

Step 11: Reporting and common mistakes

A strong model report summarizes performance as *mean ± standard deviation* across cross-validation folds, highlights at least one metric that is sensitive to class imbalance (such as balanced accuracy or macro F1), and includes a final test-set evaluation that is performed exactly once after model selection. In multiclass and imbalanced settings like this one, relying on accuracy alone can be misleading, because strong performance on a dominant class can mask weak performance on rare but important classes.

The most common mistakes in evaluation workflows are easy to describe but surprisingly common in practice: tuning on the test set, performing preprocessing outside cross-validation, reporting the best single fold instead of the mean, and declaring a “winner” when differences are smaller than the natural variability across folds. As a rule of thumb, if two models differ by less than about one standard deviation, the difference is often not practically meaningful. In those cases, simpler, faster, or more interpretable models are usually the better choice.

A subtlety worth emphasizing: the problem with evaluating on the test set is not fixed by testing more configurations—it gets worse. Each additional model you evaluate on the same test set increases the chance of selecting one that happened to score well on that particular sample. This is a form of selection bias: the more comparisons you make, the more likely you are to “discover” a model that got lucky on the test data. This is precisely why all model selection and hyperparameter tuning should happen through cross-validation on the training set, with the test set reserved for a single final evaluation.

Finally, it is worth noting that even with a disciplined test-set protocol, performance on truly new data can still differ from your test-set estimate. This can happen when the deployment data comes from a different time period, population, or environment than the data you trained on—a phenomenon

known as *distribution shift* (or dataset shift). Distribution shift is not a methodology error; it reflects a genuine change in the data-generating process. Learning curves, validation curves, and careful train/test discipline protect against problems you can control within your dataset, but monitoring model performance after deployment is necessary to detect shifts that no static evaluation can anticipate.

In the next chapter, you will extend this workflow by adding feature selection. Feature selection is still part of the modeling phase, but it introduces new opportunities for data leakage if handled incorrectly. Selecting features using the full dataset before cross-validation can bias results in the same way that tuning on the test set does. To avoid this, feature selection should be placed inside the pipeline so that it is learned separately within each training fold, preserving the evaluation discipline established in this chapter.

Summary of Workflow in Practice

The 11-step workflow above teaches the core ideas in a compact way so you can see the full process end-to-end without getting lost in excessive tuning and runtime. In practice, teams usually follow a slightly more expanded version of the workflow. The biggest difference is that *each candidate algorithm is typically tuned* (using the same cross-validation strategy and the same primary metric) *before* algorithms are compared. The table below summarizes a practical “real-world” workflow and highlights how it differs from the abbreviated version presented above.

Table 15.6
Evaluation workflow in practice (expanded) vs. the abbreviated book workflow

Practical workflow step (real projects)	What happens in practice	How the book workflow differs (and why)

Practical workflow step (real projects)	What happens in practice	How the book workflow differs (and why)
1. Define objective, constraints, and primary metric	Clarify the business decision, what errors cost, operational constraints (latency, interpretability), and choose a primary metric aligned with that decision (for example, balanced accuracy or macro F1 for imbalanced multiclass problems).	The book assumes this context so the workflow can focus on evaluation mechanics; the metric choice is emphasized, but the full requirements conversation is abbreviated.
2. Freeze a final test set early	Create a single hold-out test set (often stratified) and treat it as “do not touch” until after model selection and tuning are complete.	This is identical to the book workflow; it is one of the most important habits to build.
3. Build a leakage-safe preprocessing pipeline	Put imputation, encoding, scaling, and any other transforms inside a single pipeline so every cross-validation fold fits preprocessing on training folds only.	This is identical to the book workflow; the pipeline structure is reused throughout.

Practical workflow step (real projects)	What happens in practice	How the book workflow differs (and why)
4. Establish baseline models (multiple families)	Fit several simple baselines (for example, logistic regression, decision tree, random forest) with minimal tuning to understand the landscape.	The book includes baselines, but uses fewer candidates to keep runtime reasonable and keep the focus on the evaluation process.
5. Diagnose bias vs. variance (learning curves)	Use CV-based learning curves to decide whether more data is likely to help (variance) or whether you need a more expressive model/features (bias).	This is included in the book workflow, but typically a team would run these diagnostics for the top candidates rather than just one model.
6. Diagnose hyperparameter sensitivity (validation curves)	Use CV-based validation curves to find sensible ranges for key hyperparameters (for example, regularization strength for logistic regression).	This is included in the book workflow, but in practice it is often used to narrow ranges before larger searches across multiple parameters.

Practical workflow step (real projects)	What happens in practice	How the book workflow differs (and why)
7. Choose a shortlist of candidate algorithms	Based on baseline performance, diagnostics, constraints, and domain needs (probabilities, interpretability), pick the top 2–3 families to invest tuning effort in.	The book workflow demonstrates the idea but keeps the shortlist small to reduce complexity and keep the narrative tight.
8. Tune each candidate algorithm (same CV, same metric)	Run GridSearchCV, RandomizedSearchCV, or successive halving for each candidate using the same cross-validation strategy and the same primary metric. Use comparable compute budgets across models.	The book workflow tunes one representative model in depth to teach the method, then compares algorithms with lighter tuning to keep the example runnable in a typical notebook session.
9. Compare tuned candidates fairly	Build a comparison table of mean ± standard deviation across folds (plus training time). Treat differences within about one standard deviation as potentially unstable and use secondary criteria to break ties.	The book includes the comparison concept, but the fully tuned-per-model comparison is summarized here to keep runtime and complexity manageable.

Practical workflow step (real projects)	What happens in practice	How the book workflow differs (and why)
10. Select the final model and lock configuration	Pick the model that best balances performance, stability, and constraints. Lock hyperparameters and document the rationale (including metric tradeoffs and compute costs).	This is consistent with the book workflow; the table makes the “lock the configuration” step explicit.
11. Final test-set evaluation (once)	Refit the chosen pipeline on the full training set and evaluate on the frozen test set exactly once. Report test metrics and include per-class results (confusion matrix/classification report) for multiclass settings.	This is identical to the book workflow and is emphasized as the guardrail against accidental overfitting to the test set.
12. Reporting, monitoring plan, and next steps	Report cross-validated mean ± std, test results, and operational considerations. Identify next steps such as feature selection (next chapter), calibration (if probabilities matter), robustness checks, and monitoring for drift.	The book mentions feature selection as a next step, but the full “production handoff” items (monitoring, drift, retraining triggers) are beyond the scope of this chapter.

15.10Case Studies

Consider working through these case studies below to practice the skills taught in this chapter:

Case #1: Customer Churn (Evaluation, Selection, and Tuning)
This case uses the same **Customer Churn** dataset from earlier chapters, but shifts the focus from "training models" to a disciplined workflow for **reliable evaluation, diagnosis, hyperparameter tuning, and fair model comparison**. Your goal is to select a defensible final model using cross-validation, learning curves, validation curves, and systematic search.

Dataset attribution: Telecommunications customer churn dataset with demographics, service usage, contract attributes, and a binary churn outcome variable. See details on Kaggle.com The churn dataset is available in the prior chapter if you need to reload it.

Prediction goal: Predict whether a customer will churn (*Yes* or *No*) using all available features except the target variable.

For reproducibility, use **random_state = 27** everywhere a random seed is accepted.

Tasks

- Freeze an 80/20 stratified test set once at the beginning and do not touch it until the end.
- Build a leakage-safe preprocessing + model pipeline for a baseline logistic regression model.
- Use cross-validation to report *mean* ± *std* for accuracy, ROC AUC, F1, and log loss.
- Create a CV-based learning curve (use ROC AUC or log loss) and interpret whether the baseline shows underfitting or overfitting.

- Create a CV-based validation curve for logistic regression regularization (C) and identify a sensible tuning range.
- Tune logistic regression using *RandomizedSearchCV* (and optionally GridSearchCV in a narrow range) using the same CV object and the same primary metric.
- Compare at least three algorithms fairly (for example: tuned logistic regression, decision tree, random forest or gradient boosting) using the same CV folds and a shared scoring dictionary, and create a comparison table that includes training time.
- Evaluate the chosen final model on the frozen test set exactly once and report a confusion matrix plus at least one probability-quality metric (log loss or calibration-oriented metric if available).

Analytical questions

1. Which metric did you use as the primary selection metric, and why is it better aligned to churn decisions than accuracy alone?
2. What does the learning curve suggest: add more data, increase model capacity, or increase regularization?
3. Where does the validation curve for C peak, and what does that imply about underfitting vs overfitting?
4. Did tuning meaningfully improve performance compared to the baseline (consider mean ± std, not just the mean)?
5. Which model would you deploy and why (performance, stability, interpretability, and operational constraints)?

Customer Churn – Case Study Answers

These answers assume you used the Telco Customer Churn dataset (7,043 rows, 21 columns), froze a single 80/20 stratified test split at the start, and used the same cross-validation object and preprocessing approach across models. The goal of this case is not to “get the biggest number,” but to practice a disciplined evaluation and selection workflow that avoids leakage and produces defensible conclusions.

Q1. Primary selection metric and why it fits churn decisions

In this workflow, **ROC AUC** is a strong primary selection metric because churn interventions are typically triggered by a *risk score* or a *ranked list*, not a single fixed threshold. ROC AUC evaluates how well the model ranks churners above non-churners across all possible thresholds, which aligns with how churn teams decide whom to contact when outreach budgets are limited. Accuracy can be misleading here because the classes are imbalanced (5,174 “No” vs. 1,869 “Yes” overall), so a model can look “accurate” while still performing poorly on the churn class that matters operationally.

You also reported log loss (negative in scikit-learn’s scoring output), which complements ROC AUC by evaluating *probability quality*. If a business decision depends on calibrated probabilities (expected value, cost-sensitive thresholds, or prioritization), then log loss becomes especially important.

Q2. What the learning curve suggests

The learning curve shows a clear pattern of **high variance** (overfitting). Training ROC AUC is consistently very high (about *0.97–0.98*), while validation ROC AUC is much lower (about *0.83–0.84*) and improves only slightly as the training set grows. Even at the largest training sizes, a sizable train–validation gap remains (roughly *0.13*).

That pattern suggests that simply “adding more data” is unlikely to produce a dramatic improvement by itself (the validation curve is already flattening), and that the more promising next steps are to **increase regularization**, improve features, or move to a model family that captures nonlinear structure more effectively. Because this case intentionally focuses on evaluation and tuning (not feature engineering), the workflow appropriately moves next to regularization diagnostics and systematic search.

Q3. Where the validation curve peaks and what it implies

The validation curve for *C* peaks around **C ≈ 1**, with the highest validation ROC AUC of approximately *0.8443*. At very small *C* values (strong regularization), both training and validation ROC AUC are lower (underfitting). As *C* increases toward 1, validation performance improves. Beyond that region (for example, *C = 10* and larger), training ROC AUC rises sharply toward *1.0* while validation ROC AUC declines slightly, which is a classic sign that regularization has become too weak and the model is beginning to overfit.

This is exactly what validation curves are designed to do: identify a *reasonable region* for tuning. In this case, the curve suggests focusing search near *C* values around *0.1 to 10*, with the most defensible region centered near *1*.

Q4. Did tuning meaningfully improve performance

Tuning logistic regression did **not** meaningfully improve performance compared to the baseline when you consider mean ± standard deviation. The baseline CV ROC AUC was *0.8443 ± 0.0079*. The tuned models produced nearly identical results: *0.84429* (halving/random) and *0.84431* (random

search). Those differences are far smaller than the CV standard deviation, so they are not practically meaningful.

This is a useful outcome for students to see: sometimes the “right” conclusion is that your baseline was already near the best you can do with that model family and feature set. In that situation, the next gains typically come from a different algorithm family or improved features, not from spending more time tuning *C*.

One important detail in your outputs is that *F1* shows as *NaN* in the cross-validation summaries. This usually happens when the scorer is not configured correctly for a string-labeled target. In binary classification with labels like *Yes*/*No*, the default F1 scorer may not know which label is the “positive” class. A reliable fix is to use an explicit scorer, such as a binary F1 scorer with the positive label set to *Yes*, or to report *f1_macro* or *f1_weighted* instead. The workflow logic is still valid, but the F1 reporting should be adjusted so it reflects the churn class properly.

Q5. Which model to deploy and why

Based on the fair comparison table, **Gradient Boosting** is the most defensible choice for deployment in this case. It achieved the best cross-validated ROC AUC (*0.8485 ± 0.0089*) and the best cross-validated log loss (*0.4150 ± 0.0082*) among the tested candidates. Logistic regression was close on ROC AUC (*0.8443 ± 0.0079*) but did not surpass gradient boosting on the primary metrics.

The final frozen test-set evaluation supports that selection: the chosen gradient boosting model achieved **test ROC AUC = 0.8461** and **test log loss = 0.4166**, which are consistent with the CV estimates. The confusion matrix and classification report also highlight a realistic operational trade-off: the model

is strong at identifying non-churners (high recall for *No*), but recall for churners (*Yes*) is lower (about *0.492*). In many churn settings, that is still useful because the model's main value comes from ranking and prioritization; teams often tune the decision threshold to increase churn recall if the business prefers catching more churners at the cost of more false positives.

That said, there is also a legitimate argument for deploying logistic regression instead: it is much faster to train (about *2.3 seconds* for the CV run versus about *12 seconds* for gradient boosting in your table), easier to explain, and its performance is close. A reasonable decision rule is: choose **gradient boosting** when you want the best predictive performance and can afford the added complexity, and choose **logistic regression** when interpretability, simplicity, and maintainability dominate and the performance gap is small.

Case #2: Employee Attrition (Tuning Before Comparison)
This case study uses the **Employee Attrition** dataset (*Employee_Attrition.csv*) to practice a more realistic modeling workflow: **tune two competing algorithms first**, then compare them fairly using the same preprocessing pipeline, the same cross-validation folds, and the same primary evaluation metric. To mirror what often happens in industry, you will also compare your tuned models against an **ensemble model with default hyperparameters** to see how much performance comes "for free" with a strong off-the-shelf method.

Dataset attribution: This dataset is widely distributed as an "Employee Attrition / HR Analytics" teaching dataset based on IBM HR sample data and is provided in this course as *Employee_Attrition.csv*. See details on Kaggle.com The Employee Attrition dataset is available in the prior chapter if you need to reload it.

Prediction goal: Predict whether *Attrition* is *Yes* (employee leaves) or *No* (employee stays) using the remaining columns as predictors.

For reproducibility, use **random_state = 27** everywhere a random seed is accepted.

Primary metric guidance: Because attrition is typically a minority class, include at least one imbalance-aware metric such as *ROC AUC*, *average precision*, *balanced accuracy*, or *F1* for the attrition class. Choose one primary metric for tuning and model selection, and report additional metrics for interpretation.

Tasks

- Inspect the dataset: number of rows and columns, data types, missing values, and the class distribution of *Attrition*.
- Define *X* and *y* where *y = Attrition*. Remove identifier-style columns (for example *EmployeeNumber*) if present.
- Freeze an 80/20 stratified test set once at the beginning (*stratify=y*) and do not touch it until the end.
- Build a leakage-safe preprocessing pipeline using *ColumnTransformer*: scale numeric features with *StandardScaler* and one-hot encode categorical features with *OneHotEncoder(handle_unknown="ignore")*. Keep preprocessing inside the pipeline so it occurs inside cross-validation folds.
- Create a single cross-validation object (for example *StratifiedKFold* with 5 splits) and reuse it for all tuning and comparisons.
- **Tune Model A:** Tune a logistic regression pipeline with *RandomizedSearchCV* or *GridSearchCV* over at least *C* (and optionally

penalty if solver supports it). Record *best_params_* and the mean CV score for your primary metric.

- **Tune Model B:** Tune a tree-based model pipeline (choose one: *DecisionTreeClassifier* or *RandomForestClassifier*). Tune at least two hyperparameters (for example *max_depth* and *min_samples_leaf* for a decision tree, or *n_estimators* and *max_depth* for a random forest). Record *best_params_* and mean CV score.
- **Add an untuned ensemble baseline:** Train a gradient boosting classifier (*GradientBoostingClassifier*) or random forest with default hyperparameters inside the same preprocessing pipeline (no tuning). This will serve as a "strong default" comparison point.
- Fairly compare your three candidates (tuned Model A, tuned Model B, untuned ensemble) using the same CV folds and a shared scoring dictionary. Build a comparison table with *mean* ± *std* for your primary metric, at least one secondary metric, and CV runtime.
- Select one final model based on the comparison table and evaluate it once on the frozen test set. Report a confusion matrix and at least one probability-quality metric (*log loss* and/or *ROC AUC*, plus *average precision* if you used it).

Analytical questions

1. What is the class imbalance in *Attrition*, and why does that affect which metric you should prioritize?
2. Which metric did you choose as the primary tuning metric, and what HR decision does it best support (screening, prioritization, or probability-based planning)?
3. Did tuning logistic regression change performance meaningfully relative to its baseline (consider mean ± std, not just the mean)?

4. Did tuning the tree-based model meaningfully change performance relative to its baseline? Which hyperparameter mattered most and how could you tell?
5. How did the *untuned ensemble baseline* compare to the tuned models? What does this tell you about the value of tuning versus choosing a strong model family?
6. Which model would you recommend for deployment and why (performance, stability, interpretability, and operational constraints)?

Employee Attrition – Case Study Answers

These answers assume you used the Employee Attrition dataset, froze an 80/20 stratified test set with **random_state = 27**, and evaluated models using the same cross-validation object for fair comparison. Your results show why disciplined evaluation matters: cross-validation scores provide a stable basis for selection, while the final test set is used exactly once for a realistic estimate of deployed performance.

Q1. Dataset size and attrition rate

The dataset contains **1,470 rows** and **35 columns**, with **0 missing values** reported. The target distribution is **1,233 "No"** and **237 "Yes"**, meaning attrition occurs in about **16.12%** of employees. This class imbalance is large enough that accuracy alone can be misleading, so it is important to include metrics that focus on the minority class and probability quality (for example, ROC AUC, log loss, and F1 for *Attrition = Yes*).

Q2. Primary selection metric and why it fits HR decisions

The primary selection metric in this workflow was **ROC AUC**. ROC AUC is often better aligned with HR attrition decisions than accuracy because attrition interventions are usually triggered by *risk scores* (ranking employees by likelihood of leaving) rather than a single hard cutoff. ROC AUC measures how well the model ranks “Yes” cases ahead of “No” cases across all possible thresholds, which makes it robust when the positive class is relatively rare. In addition, the workflow tracked **log loss** to evaluate probability quality and **F1 for Attrition = Yes** to ensure the model does not ignore the high-risk group.

Q3. Baseline performance from cross-validation

The baseline logistic regression model produced strong cross-validated accuracy (**0.8818 ± 0.0130**) and reasonable ranking performance (**ROC AUC = 0.8420 ± 0.0424**). The F1 score for the attrition class (**0.5409 ± 0.0690**) confirms that the model is capturing meaningful signal for the minority class, though there is noticeable fold-to-fold variability (the standard deviation is not trivial). The cross-validated log loss of **0.3229 ± 0.0354** suggests the predicted probabilities are reasonably informative, which matters if HR uses probabilities to prioritize interventions.

Q4. Did tuning improve performance in a meaningful way?

Tuning logistic regression with randomized search selected **C = 0.1728** and achieved **ROC AUC = 0.8434** in cross-validation for the best configuration. In the comparison table, the tuned logistic regression slightly improved accuracy (**0.8852 ± 0.0118**) and improved log loss (**0.3184 ± 0.0303**) relative to the baseline values (*0.8818 ± 0.0130* accuracy; *0.3229 ± 0.0354* log loss). However, the improvement is small relative to the cross-validation variability, so the most defensible conclusion is that tuning provided a *modest* gain in

probability quality and overall fit rather than a dramatic performance jump. This is a common outcome when a baseline model is already competitive.

Q5. Comparing tuned algorithms and an untuned ensemble

The tuned logistic regression model was the best overall performer in the CV comparison: **ROC AUC = 0.8434 ± 0.0428**, **log loss = 0.3184 ± 0.0303**, and **accuracy = 0.8852 ± 0.0118**. The untuned gradient boosting model was competitive but weaker on all tracked metrics (*ROC AUC = 0.8099 ± 0.0347*, *log loss = 0.3436 ± 0.0231*, *accuracy = 0.8708 ± 0.0143*). The tuned decision tree performed worst on ranking and probability quality (*ROC AUC = 0.7354 ± 0.0494*, *log loss = 1.5683 ± 0.3346*), which is consistent with single trees producing unstable probability estimates. These results illustrate an important modeling lesson: an untuned ensemble is not guaranteed to beat a well-tuned, well-regularized baseline.

Q6. Test-set evaluation of the chosen model

Using the frozen test set exactly once, the chosen final model was **tuned logistic regression**. On the test set, it achieved **accuracy = 0.8810**, **ROC AUC = 0.7990**, and **log loss = 0.3417**, with **PR AUC = 0.6152**. The confusion matrix shows **20 true positives** and **27 false negatives** for attrition (the model correctly identified 20 employees who left but missed 27 who left). The classification report confirms this tradeoff: precision for *Yes* is **0.7143**, but recall is **0.4255**. In HR contexts where missing high-risk employees is costly, this result can motivate threshold tuning, class-weighting, or alternative metrics—while still preserving the leakage-safe workflow.

Q7. Deployment recommendation

A defensible deployment recommendation is to use the **tuned logistic regression** model. It provided the best cross-validated ranking performance and probability quality among the compared options, trained quickly, and remains highly interpretable (coefficients can be explained and audited). The key operational caveat is recall for the attrition class: the test-set recall of **0.4255** indicates that many true attrition cases are still missed. In practice, HR teams often address this by adjusting the decision threshold to prioritize recall, using cost-sensitive decision rules, or combining the model with targeted business processes (for example, follow-up surveys or manager reviews) for the highest-risk segment.

Case #3: Telco Support Ticket Priority (Evaluation, Tuning, and Deployment Decisions)

This case uses a Telco customer support dataset of service tickets. Your goal is to build a disciplined workflow for **reliable evaluation, hyperparameter tuning, and fair comparison** of multiple multiclass classifiers that predict ticket *priority* (*Low*, *Medium*, *High*). In a real support organization, the cost of misclassifying *High*-priority tickets is often much higher than the cost of confusing *Low* and *Medium*. This case emphasizes both threshold-based performance (*accuracy*, *macro F1*) and probability quality (*multiclass log loss*).

Dataset attribution: The dataset file for this case is *Support_tickets.csv*. See details on Kaggle.com The Support Ticket Priority dataset is available in the prior chapter if you need to reload it.

Prediction goal: Predict *priority* (*Low*, *Medium*, *High*). Treat the classes as *nominal* (not ordered) and use standard multiclass classification metrics.

Recommended feature set: Use a mix of numeric and categorical predictors. Prefer human-readable categorical columns (for example: *day_of_week*,

company_size, *industry*, *customer_tier*, *region*, *product_area*, *booking_channel*, *reported_by_role*, *customer_sentiment*) and numeric operational columns (for example: *org_users*, *past_30d_tickets*, *past_90d_incidents*, *customers_affected*, *error_rate_pct*, *downtime_min*, plus binary flags such as *payment_impact_flag* and *security_incident_flag*). Exclude identifier-like columns such as *ticket_id*.

Note on duplicate encodings: This dataset includes both readable categorical columns and numeric-coded versions (for example, *industry* and *industry_cat*). Use only one representation. The recommended approach is to use the readable categorical columns with one-hot encoding and drop any *_cat* columns.

For reproducibility, use **random_state = 27** everywhere a random seed is accepted.

Tasks

- Inspect the dataset: report number of rows and columns, list the unique values of *priority*, and compute class counts and percentages for *Low*, *Medium*, and *High*.
- Create *X* and *y* where *y = priority*. Remove *ticket_id* and drop all columns whose names end with *_cat*.
- Freeze a single 80/20 train/test split using *random_state=27* and *stratify=y*. Do not touch the test set until the end.
- Build a leakage-safe preprocessing pipeline using *ColumnTransformer*: scale numeric predictors with *StandardScaler*, one-hot encode categorical predictors using *OneHotEncoder(handle_unknown="ignore")*, and fit all preprocessing *inside* cross-validation via a pipeline.

- Create a baseline classifier that always predicts the most frequent priority class. Report test-set *accuracy* and multiclass *log loss* using class proportions as constant probability predictions.
- Choose a single cross-validation strategy (for example, *StratifiedKFold*) and a primary tuning metric (recommend *neg_log_loss* for probability-based decisions). Use the *same CV object* and *same primary metric* consistently for all tuned models.
- Tune two non-ensemble algorithms using *RandomizedSearchCV* (or a narrow *GridSearchCV* if computationally feasible). Recommended pair: multinomial logistic regression and an SVM classifier (or k-NN). Report the best hyperparameters and the mean cross-validated primary score for each tuned model.
- Tune two ensemble algorithms using the same CV object and primary metric. Recommended pair: *RandomForestClassifier* and *GradientBoostingClassifier* (or AdaBoost). Use provided hyperparameter ranges (below) and record cross-validated results.
- Train one additional ensemble model *without tuning* (for example, bagging or AdaBoost with default-like settings). Evaluate it using the same CV object and scoring dictionary so it can be compared fairly.
- Build a single comparison table that includes: *Model*, *Accuracy (mean ± std)*, *Macro F1 (mean ± std)*, *Log Loss (mean ± std)*, and *CV time*. Sort by your primary metric.
- Select the best two candidates based on your comparison table and evaluate them on the frozen test set exactly once. Report test-set accuracy, macro F1, log loss, and confusion matrices. Identify the most common misclassifications.
- Create two bar charts: one comparing test-set (or CV-mean) *accuracy* across all models, and one comparing *log loss* across all models. Use these charts to support a deployment recommendation.

- Write a short deployment checklist (5–8 bullets) describing how you would choose a model in a real support organization, considering probability quality, minority-class behavior (especially *High* priority), interpretability, and runtime constraints.

Suggested tuning ranges

- **Logistic regression (multinomial):** *C* on a log scale (for example, 1e-3 to 1e3), *l1_ratio* (0.0 for L2, 1.0 for L1, or values in between for elastic net), and solver *saga* which supports all *l1_ratio* values and multinomial classification.
- **SVM (if used):** *C* on a log scale; if using an RBF kernel, tune *gamma* on a log scale; consider probability outputs only if your implementation supports them reliably.
- **k-NN (if used):** *n_neighbors*, distance weighting (uniform vs distance), and a small set of distance metrics supported by your environment.
- **Random forest:** *n_estimators* (for example, 200–800), *max_depth* (None or integers such as 5–30), *min_samples_leaf* (1–20), and *max_features* (sqrt, log2, or fractions if supported).
- **Gradient boosting:** *n_estimators* (for example, 100–500), *learning_rate* (for example, 0.01–0.2), and *max_depth* (for example, 2–5). Tune cautiously because some combinations can overfit.
- **AdaBoost (if used):** *n_estimators* (for example, 50–400) and *learning_rate* (for example, 0.01–1.0). If using tree stumps, confirm the base estimator depth is small.

Analytical questions

1. What are the class proportions for *Low*, *Medium*, and *High*, and how does that influence your choice of metrics?

2. How does the baseline model perform in terms of accuracy and log loss, and why is log loss especially informative in multiclass settings?
3. Which tuned model achieved the lowest cross-validated log loss? Did it also achieve the best accuracy or macro F1?
4. Which class appears hardest to predict correctly, and what evidence supports your claim (macro F1 behavior, per-class precision/recall, confusion matrices)?
5. Compare random forests and bagging. Which performs better, and what does that suggest about variance reduction and feature sampling in this dataset?
6. Did boosting improve probability quality (log loss) relative to bagging and random forests? If so, what tradeoffs did you observe in training time or interpretability?
7. Did stacking improve performance over the best single tuned model? Support your conclusion using at least two metrics and discuss whether the improvement appears practically meaningful.
8. Which two priority levels are most frequently confused in the confusion matrices? Propose one operational reason that might explain this confusion.
9. Decision question (5–8 sentences): Which model would you deploy in a real support operation and why? Address (1) the cost of misclassifying *High* priority tickets, (2) probability quality, (3) stability (mean ± std), and (4) runtime constraints.
10. Deployment checklist: Provide 5–8 bullet points describing the practical steps you would take after model selection (monitoring, threshold policy, feedback loops, drift checks, and periodic retraining).

Telco Support Ticket Priority – Case Study Answers

These answers assume you used the Support Ticket Priority dataset, removed identifier-style columns (for example *ticket_id*) and duplicate encoded *_cat* columns, created an 80/20 stratified train/test split with **random_state = 27**, and evaluated models with the same preprocessing pipeline and consistent cross-validation folds. Values referenced below come directly from the results you reported (CV means ± std, plus the final frozen test evaluation for the chosen model).

Q1. Class proportions and metric choice

The class distribution is **Low = 50%** (25,000 of 50,000), **Medium = 35%** (17,500), and **High = 15%** (7,500). Because the classes are not equally common, *accuracy* can be misleading: a model can look "good" by doing very well on the majority class (*Low*) while still making many mistakes on the more consequential minority class (*High*). This is why you included **macro F1** (treats each class equally) and **multiclass log loss** (measures probability quality for all classes, not just the top prediction). In ticket routing, models often drive queueing and escalation decisions, so probability quality matters —not just whether the single top label is correct.

Q2. Baseline performance and why log loss matters

The baseline "most frequent class" model achieved **test accuracy = 0.50** because it always predicts *Low*, which is 50% of tickets. However, its probability-based performance was poor: using class proportions as constant probabilities produced **test log loss = 0.9986**. Log loss is especially informative in multiclass settings because it penalizes *overconfident wrong probabilities* and rewards models that assign higher probability to the correct

class even when they sometimes miss the top label. In support operations, probabilities are useful for downstream decisions (routing confidence, escalation thresholds, SLA risk), so a model with lower log loss is often more operationally valuable than one that merely improves accuracy.

Q3. Best tuned model by cross-validated log loss

The tuned model with the lowest cross-validated log loss was **GBDT (tuned)** with **CV log loss = 0.0808 ± 0.0018**. It also achieved the best overall performance on the other reported CV metrics: **accuracy = 0.9674 ± 0.0002** and **macro F1 = 0.9619 ± 0.0004**. In other words, the model that was best on probability quality (log loss) was also best on label quality (accuracy and macro F1) in this experiment, which is not always true—but it is an ideal outcome when it happens.

Q4. Hardest class to predict (evidence)

The hardest class to predict correctly is **High**. The strongest evidence comes from the final test evaluation of the best model (GBDT tuned): **High recall = 0.9393** is the lowest recall among the three classes (compared to *Low recall = 0.9824* and *Medium recall = 0.9657*). The confusion matrix shows the dominant error pattern: **High tickets predicted as Medium (91 cases)**, while *High predicted as Low (0)* was essentially eliminated. This is a common operational pattern: “High vs Medium” is often a subtler distinction than “High vs Low,” so most mistakes cluster between adjacent urgency levels even when the classes are nominal.

Q5. Random forests vs bagging

In your comparison table, **Random Forest (tuned)** substantially outperformed the other variance-reduction-style approach you included, with **CV log loss = 0.3275 ± 0.0012** and **macro F1 = 0.8885 ± 0.0006**. Conceptually, this result is consistent with why random forests are often stronger than plain bagging: both reduce variance by averaging many trees, but random forests also reduce *correlation* among trees through **feature subsampling** (here, your best settings included *max_features = sqrt*). When trees are less correlated, the ensemble average is more stable and generalizes better. Practically, this suggests the dataset contains multiple predictive feature "paths," and forcing trees to explore different subsets helps.

Q6. Boosting vs bagging/random forests (probability quality + tradeoffs)

Yes—boosting dramatically improved probability quality. The tuned GBDT achieved **CV log loss = 0.0808**, far better than the tuned random forest (*0.3275*) and far better than the untuned AdaBoost you included (*1.0154*). The key tradeoff you observed was **training time**: GBDT was the slowest to tune (your tuning time was on the order of thousands of seconds, and the CV evaluation time for the tuned GBDT was also far larger than the other models). Interpretability also shifts: boosted tree ensembles can be explained (feature importance, SHAP), but they are not as immediately transparent as a single decision tree or logistic regression. So the practical decision is often: if the support organization needs highly reliable probabilities for routing/escalation, GBDT is compelling; if runtime and simplicity dominate, a simpler model may be preferred.

Q7. Did stacking improve over the best single tuned model?

No—stacking did not improve performance over the best single tuned model. Your best tuned model (GBDT) achieved **CV log loss = 0.0808**, **accuracy = 0.9674**, and **macro F1 = 0.9619**. None of the other evaluated models came close on log loss, and even the next-best (random forest tuned) was much worse on probability quality (*0.3275*) and also lower on accuracy and macro F1. When one model class is already capturing most of the predictive structure, stacking often adds complexity without gains—especially if the base learners are not complementary enough or if the problem is already well solved by a single strong approach. Practically, the improvement would need to be both measurable and meaningful relative to operational costs; here it was not.

Q8. Most frequent confusions and an operational explanation

The most frequent confusion pair was **High predicted as Medium** (91 cases in the test confusion matrix for the best model). Other common confusions were *Low predicted as Medium* (87) and *Medium predicted as Low* (79), with a smaller number of *Medium predicted as High* (41). One operational reason is that the boundary between High and Medium urgency often depends on context that may be only partially represented in structured fields (for example, the true business impact, severity perceived by the customer, or hidden technical root cause). If the features strongly represent “something is wrong” but not the full escalation context, the model will correctly avoid labeling those tickets as Low, but still struggle to decide High vs Medium consistently.

Q9. Deployment decision

I would deploy **GBDT (tuned)**. First, it best protects against costly mistakes involving *High* priority tickets: its test results show strong High-class

performance (precision *0.9711*, recall *0.9393*), and most of its remaining High errors are “High → Medium,” not “High → Low,” which is operationally safer. Second, it has the best probability quality: **test log loss = 0.0691** and the lowest CV log loss (*0.0808 ± 0.0018*), which matters if you use probabilities to drive routing confidence, queue prioritization, or escalation thresholds. Third, its cross-validation variability is extremely small (tiny std values), suggesting stable performance across folds. The main concern is runtime: GBDT was the slowest to tune and slower to cross-validate than simpler models, so I would confirm inference latency meets ticket-volume constraints and consider operational tactics (batch scoring, model serving optimization, or slightly smaller hyperparameter settings) if needed.

Q10. Deployment checklist

- Define an escalation policy: decide how predicted probabilities translate into routing rules (for example, auto-escalate when *P(High)* exceeds a chosen threshold).
- Validate minority-class behavior: confirm minimum recall targets for *High* priority tickets on a recent holdout slice before launch.
- Check probability calibration: evaluate log loss on recent data and recalibrate probabilities if they drift (calibration matters for threshold-based routing).
- Stress-test by segment: measure performance by region, product area, customer tier, and channel to ensure robustness (and to detect systematic failure modes).
- Measure runtime constraints: benchmark inference latency and cost at expected ticket volume; choose batch vs real-time scoring accordingly.
- Set monitoring dashboards: track drift, log loss, class distribution, and confusion patterns over time with alert thresholds.

- Create a feedback loop: capture corrections from agents (true priority) and feed them into periodic retraining and post-deployment backtesting.
- Schedule retraining: retrain on a fixed cadence (or when drift triggers fire), and re-validate the threshold policy after each refresh.

15.11 Assignment

Complete the assignment below.

This assessment can be taken online.